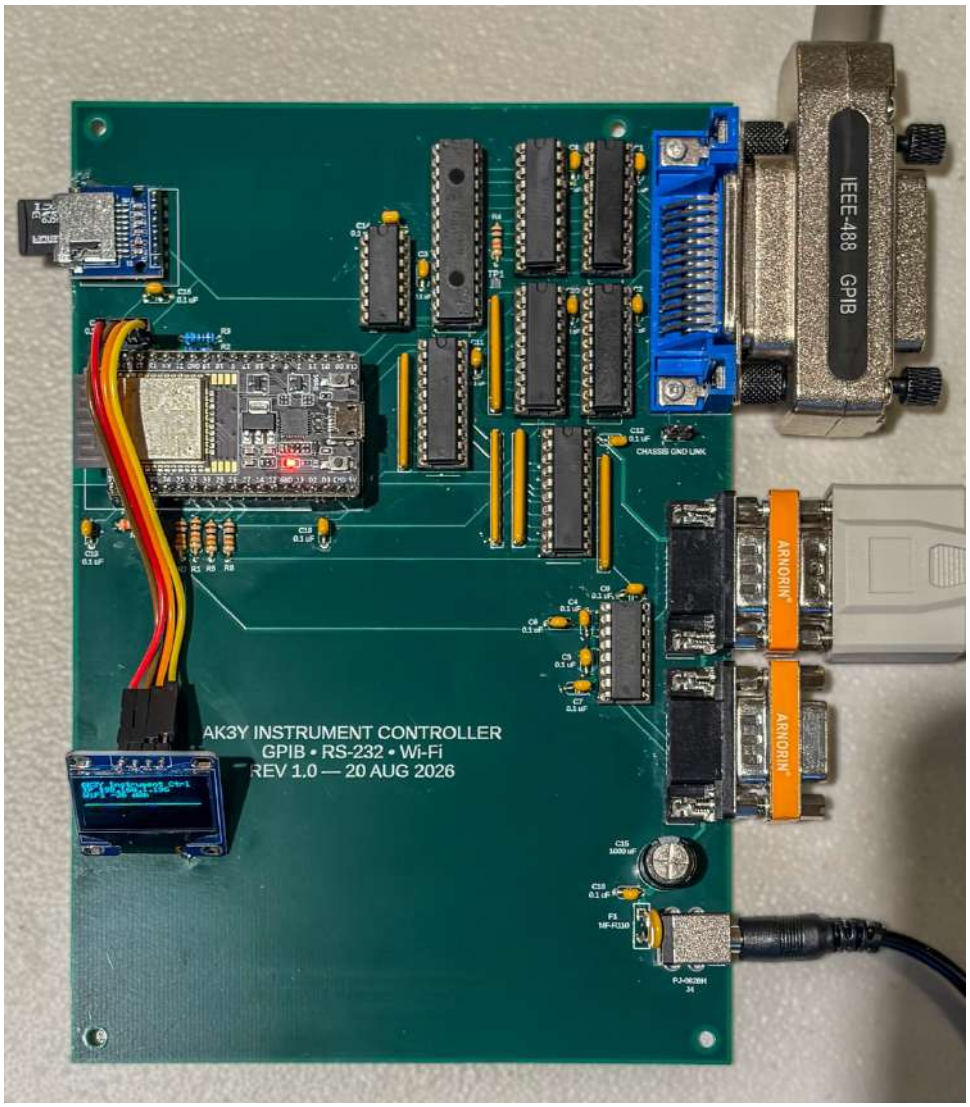
AK3Y Instrument Controller
User's Manual



Revised 25 October 2026

Table of Contents

1. Introduction 1
1.1 Overview 1
1.2 System Capabilities 1
1.3 Instrument Profiles and Macros 2
1.4 Organization of This Manual 3
2. Overview of the Web Interface 4
2.1 Instruments Screen 4
2.2 Macros Screen 6
2.3 File Management Screen 8
2.4 About Screen 10
2.5 Firmware Update Screen 11
2.6 Update Wi-Fi Screen 12
3. Software Installation and Initial Startup 13
3.1 Introduction 13
3.2 Obtaining the Software 13
3.3 Building and Installing the Firmware Using PlatformIO 14
3.3.1 Development Environment 14
3.3.2 Installing PlatformIO 14
3.3.3 Obtaining the Source Code 14
3.3.4 Compiling the Firmware 14
3.3.5 Programming the ESP32 15
3.3.6 Monitoring Serial Output 15
3.4 Installing Precompiled Firmware 15
3.5 Preparing the microSD Card 16
3.6 Initial Startup and Wi-Fi Configuration 16
3.7 Verifying the Installation 17
3.8 Installing Instrument Definitions and Profiles 18
3.9 Saving an Initial Workspace 18
4. Configuring and Operating Instruments 20

4.1 Introduction 20
4.2 Instrument Definitions and Profiles 20
4.3 Creating an Instrument Control Panel 21
4.4 Understanding the Control Panel 22
4.5 Setting and Reading Instrument Parameters 23
4.5.1 Setting a Numerical Value 23
4.5.2 Reading a Parameter 23
4.5.3 Engineering Units 24
4.5.4 Reading Versus Setting 24
4.6 Selecting Instrument Functions 25
4.6.1 Selecting Basic RF Generator 25
4.6.2 Selecting Carrier Frequency Sweep 25
4.6.3 Action Buttons 27
4.7 Operating Multiple Instruments 27
4.8 Using the Advanced Command Console 28
4.9 Saving and Restoring Control Panel Arrangements 28
4.10 Communication Errors and Troubleshooting 29
4.11 Summary 30
5. Creating and Executing Macros 31
5.1 Introduction 31
5.2 Macro Files 31
5.3 Selecting a Macro 32
5.4 Example 1: FrequencySweep 32
5.4.1 Macro Description 32
5.4.2 Input Parameters 33
5.4.3 Macro Instructions 33
5.4.4 Measurement Results 35
5.5 Example 2: FindSignal 36
5.5.1 Input Parameters 36
5.5.2 Defining Displayed Results 37
5.5.3 Locating the Strongest Signal 37

5.5.4 Understanding the Measurement 38
5.6 Macro Variables and Parameters 39
5.7 Executing and Monitoring a Macro 39
5.8 Macro Errors and Troubleshooting 40
5.9 Summary 40
6. Workspace Management 42
6.1 Introduction 42
6.2 The Workspace Configuration Controls 42
6.3 Creating a Workspace 43
6.4 Saving Changes to an Existing Workspace 44
6.4.1 Unsaved Changes and Reverting a Configuration 44
6.5 Loading a Saved Workspace 45
6.6 Renaming and Deleting Workspaces 46
6.6.1 Renaming a Workspace 46
6.6.2 Deleting a Workspace 46
6.7 Selecting a Startup Workspace 46
6.8 Workspace Files on the microSD Card 47
6.9 Workspaces and Macro Panels 48
6.10 Practical Considerations 48
6.11 Summary 49
7. File Management and System Maintenance 50
7.1 Introduction 50
7.2 Organization of the microSD Card 50
7.3 Creating Directories 51
7.4 Uploading Files 51
7.4.1 Selecting a File 51
7.4.2 Upload Size Limit 52
7.4.3 Replacing an Existing File 52
7.4.4 When Do Uploaded Changes Take Effect? 52
7.5 Viewing and Editing Files 52
7.5.1 Viewing a File 53

7.5.2 Editing a File 53
7.6 Downloading and Deleting Files 53
7.6.1 Downloading a File 53
7.6.2 Deleting a File 54
7.6.3 Deleting Directories 54
7.7 Diagnostic and Measurement Log Files 54
7.7.1 Controller Diagnostics 54
7.7.2 Measurement Files 55
7.7.3 Managing Log Files 55
7.8 Backing Up and Restoring the microSD Card 56
7.9 File Management Errors and Recovery 57
7.10 Summary 58
8. Network Configuration, Firmware Updates, and Troubleshooting 59
8.1 Introduction 59
8.2 Normal Network Operation 59
8.2.1 Network Requirements 60
8.2.2 Network Status Information 60
8.3 Changing the Wi-Fi Network 60
8.3.1 Entering Wi-Fi Setup Mode 61
8.3.2 Connecting to the Setup Access Point 61
8.3.3 Saving the New Network Configuration 61
8.3.4 Setup Timeout and Cancellation 62
8.3.5 Storage of Wi-Fi Credentials 62
8.4 Firmware Updates 62
8.4.1 Obtaining a Firmware Image 62
8.4.2 Checking the Installed Firmware 63
8.4.3 Installing an Update 63
8.4.4 Verifying the Update 63
8.4.5 Firmware Update Problems 64
8.5 Using the About Screen for Diagnostics 64
8.5.1 Controller Uptime 65

8.5.2 Available Memory 66
8.5.3 microSD Card Status 66
8.6 Troubleshooting Network Problems 66
8.6.1 Controller Cannot Be Reached from the Browser 66
8.6.2 Controller Does Not Connect to Wi-Fi 66
8.6.3 Browser Connection Is Intermittent 67
8.6.4 TCP Instruments Cannot Be Reached 67
8.7 Troubleshooting Instrument Communications 67
8.7.1 GPIB Instruments 67
8.7.2 RS-232 Instruments 68
8.7.3 TCP Instruments 68
8.7.4 Instrument Profile Errors 68
8.8 Troubleshooting Macros and Workspaces 69
8.8.1 Macro Does Not Appear 69
8.8.2 Macro Execution Fails 69
8.8.3 Saved Workspace Does Not Match Instrument Settings 69
8.8.4 Unsaved Workspace Changes 70
8.9 Diagnostic Logs and Problem Reporting 70
8.10 Troubleshooting Summary 71
8.11 Summary 72
Appendix A A-1
A.1 Purpose and file location A-1
A.2 Organization and format of the configuration file A-1
A.3 Parameters required for every instrument A-2
A.4 GPIB instruments A-2
A.5 RS-232 instruments A-3
A.6 TCP network instruments A-3
A.7 Complete configuration example A-4
A.8 Installing and checking the file A-5
A.9 Configuration file limitations A-5
A.10 Alphabetical parameter reference A-5

Appendix B B-1
B.1 Introduction B-1
B.2 A Simple Instrument Profile: Marconi 2024 ..... B-1
B.2.1 The [instrument] Section B-2
B.2.2 The [frequency] Section B-3
B.2.3 The [level] Section B-5
B.2.4 What the Controller Displays B-6
B.3 Adding a Frequency Sweep Function B-6
B.3.1 Defining a Function Group B-7
B.3.2 Numerical Sweep Controls B-8
B.3.3 Selection Controls: `type=select` B-9
B.3.4 Conditional Controls: `visible_when` B-10
B.3.5 Response Units B-11
B.3.6 Action Buttons: `type=action` B-11
B.3.7 Two-State Action Buttons: `type=toggle_action` B-12
B.3.8 Hidden Capabilities: `ui=false` B-12
B.3.9 The Completed Sweep Function B-13
B.4 Instrument Profile Reference Tables B-15
Appendix C C-1
C.1 Introduction C-1
C.2 Macro File Organization C-2
C.2.1 The [macro] Section C-2
C.2.2 Input Parameters C-3
C.2.3 Displayed Results C-4
C.2.4 The [steps] Section C-4
C.3 Macro Input Parameter Types C-5
C.3.1 Numerical Parameters: type=number C-5
C.3.2 Boolean Parameters: type=boolean C-6
C.3.3 Selection Parameters: type=select C-7
C.3.4 Input Parameter Summary C-7
C.4 Instrument Commands C-8

C.4.1 SET — Set an Instrument Parameter C-8
C.4.2 QUERY — Read an Instrument Parameter C-9
C.4.3 ACTION — Execute an Instrument Action C-9
C.4.4 ACTIONON — Execute an Alternate Action C-10
C.4.5 ENTER — Enter an Instrument Function C-10
C.4.6 WAIT — Check Instrument Readiness C-10
C.5 Macro Control Commands C-11
C.5.1 DELAY — Pause Execution C-11
C.5.2 SWEEP — Begin a Numerical Loop C-11
C.5.3 ENDSWEEP — End a Numerical Loop C-12
C.5.4 LOG — Record Measurement Results C-13
C.6 Variables and Substitution C-13
C.6.1 Input Parameter Substitution C-13
C.6.2 Runtime Variables C-14
C.6.3 Runtime Arithmetic C-14
C.6.4 Substitution Errors C-15
C.7 Macro Command Summary C-15
C.8 Macro File Parameters C-16
C.9 Programming Limits and File Conventions C-16
C.9.1 Comments and Blank Lines C-17
C.9.2 Capitalization and Spacing C-17
C.9.3 File Length and Formatting C-17
C.10 Developing and Testing Macros C-18
C.11 Summary C-19
Appendix D D-1
D.1 Introduction D-1
D.2 Command Format D-1
D.2.1 Supported Commands D-2
D.3 Setting Instrument Parameters — `SET` D-2
D.4 Reading Instrument Parameters — `QUERY` D-3
D.5 Executing Instrument Actions — `ACTION` D-5

D.6 Executing Alternate Toggle Actions — ACTIONON D-6
D.7 Entering Instrument Functions — ENTER D-6
D.7.1 Entering the Basic Function D-6
D.7.2 Entering Other Instrument Functions D-7
D.7.3 Additional Considerations D-8
D.8 Waiting for Instrument Readiness — WAIT D-8
D.9 Practical Console Examples D-9
D.9.1 Testing the Marconi 2024 ..... D-9
D.9.2 Testing the E4407B Spectrum Analyzer D-10
D.9.3 Testing a Sweep Action D-11
D.10 Console Command Syntax Summary D-11
D.11 Error Messages and Troubleshooting D-12
D.12 Relationship to Macros D-13
D.13 Summary D-14

1. Introduction

1.1 Overview

The AK3Y Instrument Controller is a compact, microprocessor-based system designed to provide computer control of electronic test and measurement instruments. The controller is built around the Espressif ESP32-DevKitC-32E development board, which incorporates an ESP32-WROOM-32E module with a dual-core 32-bit Xtensa LX6 processor operating at up to 240 MHz.

The controller supports instruments equipped with IEEE-488 (GPIB), RS-232, and Ethernet (TCP) interfaces. Its integrated Wi-Fi capability provides browser-based instrument control without requiring dedicated software on the user's computer.

The controller was developed to provide a flexible and economical means of operating laboratory instruments, particularly older equipment that remains useful but lacks modern computer interfaces. Many such instruments incorporate GPIB or RS-232 connections and can be controlled remotely using manufacturer-defined command sets. The AK3Y Instrument Controller makes these capabilities accessible through a standard web browser, without requiring dedicated instrument-control software on the user's computer.

In addition to direct instrument control, the system supports automated test sequences, parameterized macros, measurement data collection, and storage of instrument definitions and configuration files on a microSD card.

The controller is intended for amateur radio experimenters, electronics enthusiasts, and others who wish to automate laboratory measurements or integrate instruments from different manufacturers into a common control environment.

1.2 System Capabilities

The principal capabilities of the AK3Y Instrument Controller include:

Multiple instrument interfaces. Support for IEEE-488 (GPIB), RS-232, and TCP-based instrument communications.

Browser-based operation. Instrument control and system management through a web interface accessible from a computer or other suitable device on the local network.

Instrument-specific controls. Configurable graphical Control Panels that present the functions and parameters defined for individual instruments.

Multiple simultaneous Control Panels. Independent panels for selecting and operating different instruments from a single browser window.

Automated measurements. Execution of macros that coordinate instrument commands, parameter changes, and measurement operations.

Parameterized macros. User-adjustable input parameters and display of results and execution progress.

Workspace configurations. Saving and restoring Control Panel arrangements, instrument selections, and displayed parameter values.

microSD card storage. Storage and browser-based management of instrument profiles, macros, configuration files, and diagnostic logs.

Wireless connectivity. Wi-Fi network operation with browser-based network reconfiguration.

Firmware maintenance. Over-the-air firmware updates without requiring a direct USB connection.

Together, these capabilities permit both simple remote instrument operation and more sophisticated automated measurement procedures.

1.3 Instrument Profiles and Macros

Two important features distinguish the AK3Y Instrument Controller from a simple interface adapter: instrument profiles and macros.

An *instrument profile* defines the commands, functions, and operating parameters associated with a particular instrument. These definitions are stored on the microSD card and used to generate the corresponding browser-based controls. The operator can therefore select an instrument and work with labeled controls rather than repeatedly entering low-level commands.

A *macro* defines an automated sequence of operations. Macros can be used for repetitive instrument adjustments, measurements over a specified frequency range, or procedures involving more than one instrument.

For example, a signal generator can be stepped through a range of frequencies while a spectrum analyzer measures the resulting signal. A more elaborate macro can search a frequency range, identify the strongest signal, and report its frequency and amplitude.

Because instrument profiles and macros are stored separately from the firmware, the controller can be adapted to additional instruments and measurement procedures without recompiling the application.

1.4 Organization of This Manual

This manual describes the operation and configuration of the AK3Y Instrument Controller.

Chapter 2 introduces the browser interface and provides an overview of its six principal screens. Subsequent chapters describe instrument configuration and operation, macro execution and programming, workspace management, microSD card file operations, network configuration, firmware updates, and troubleshooting.

The manual is intended primarily as an operating reference. Detailed discussions of the controller's hardware design and firmware implementation are presented separately in the accompanying technical article and project documentation.

2. Overview of the Web Interface

The AK3Y Instrument Controller provides a browser-based interface for operating electronic test equipment connected through IEEE-488 (GPIB), RS-232, and LAN (TCP) interfaces. Users can configure instruments, adjust operating parameters, execute automated measurement sequences, and manage files stored on the controller's microSD card.

The interface is organized into six principal screens, accessible through navigation buttons at the top of the browser window. The Instruments, Macros, File Management, and About screens share a common navigation and status header. Separate pages provide firmware updating and Wi-Fi configuration.

The status header displays the Wi-Fi connection and signal strength, SD card status, clock synchronization status, and installed firmware version.

2.1 Instruments Screen

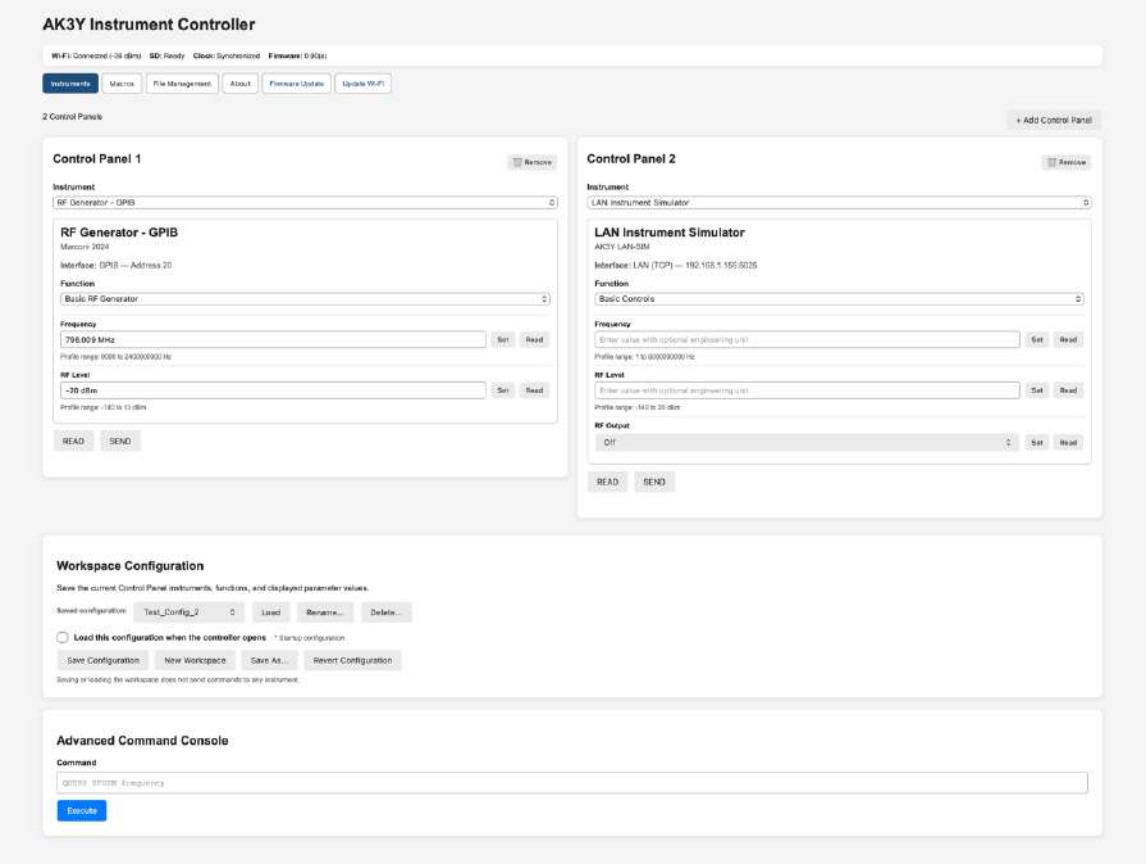


Figure 2-1. Instruments screen showing two Control Panels and the workspace configuration controls.

The Instruments screen is the primary interface for direct instrument operation. Each Control Panel represents a selected instrument and provides controls appropriate to that instrument's capabilities. Multiple Control Panels can be displayed simultaneously, allowing the operator to work with several instruments without switching between separate browser pages.

In Figure 2-1, Control Panel 1 is configured for a Marconi 2024 RF signal generator connected through GPIB at address 20. Its controls include frequency and RF output level. Control Panel 2 is configured for a LAN Instrument Simulator communicating over TCP.

Instrument-specific functions are selected from a dropdown menu. Depending on the selected function, the panel presents editable parameters, numerical values, selection controls, and Set or Read buttons.

Below the Control Panels are two additional sections:

> Workspace Configuration allows the operator to save, load, rename, delete, and select a startup configuration. A workspace configuration preserves the arrangement of Control Panels, their instrument selections, selected functions, and displayed parameter values. Loading a configuration does not itself transmit commands to connected instruments.
>
> Advanced Command Console provides a means of entering and executing commands directly, without using the graphical instrument controls.

Control Panels can be added or removed as needed.

2.2 Macros Screen

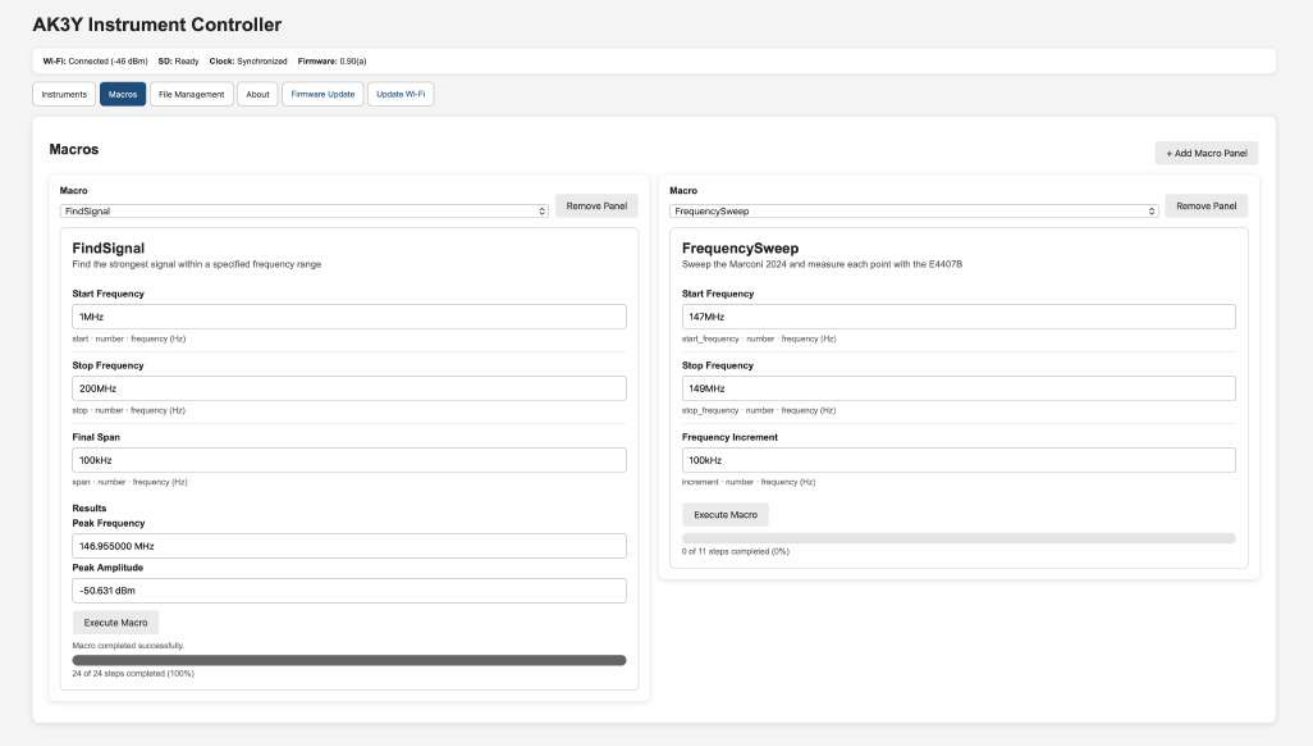


Figure 2-2. Macros screen showing a completed signal-search operation and a frequency-sweep macro.

The Macros screen provides access to automated instrument operations.

Macros are predefined sequences of commands and operations stored on the controller's microSD card. They allow repetitive measurements, instrument adjustments, and more complex test procedures to be performed automatically.

The interface supports multiple Macro Panels, each containing a selectable macro and its associated controls.

The screenshot illustrates two examples:

> The macro *FindSignal* searches a specified frequency range for the strongest signal. In this case, the signal was generated by the Marconi 2024 while being received and processed by an Agilent E4407B spectrum analyzer. The example shows a completed search reporting a peak frequency of 146.955 MHz and a measured peak amplitude of −50.631 dBm.
>
> The second macro, *FrequencySweep* sweeps the Marconi 2024 signal generator in steps over a specified frequency range while taking measurements with the E4407B.

Macro input parameters appear as labeled fields. In the examples, these include starting frequency, stopping frequency, frequency increment, and final search span.

Each Macro Panel includes an Execute Macro button and a progress indicator showing the number of completed steps. Where supported by the macro definition, measurement results will also appear directly within the panel.

Multiple Macro Panels allow different automated procedures to remain available in the same browser session.

2.3 File Management Screen

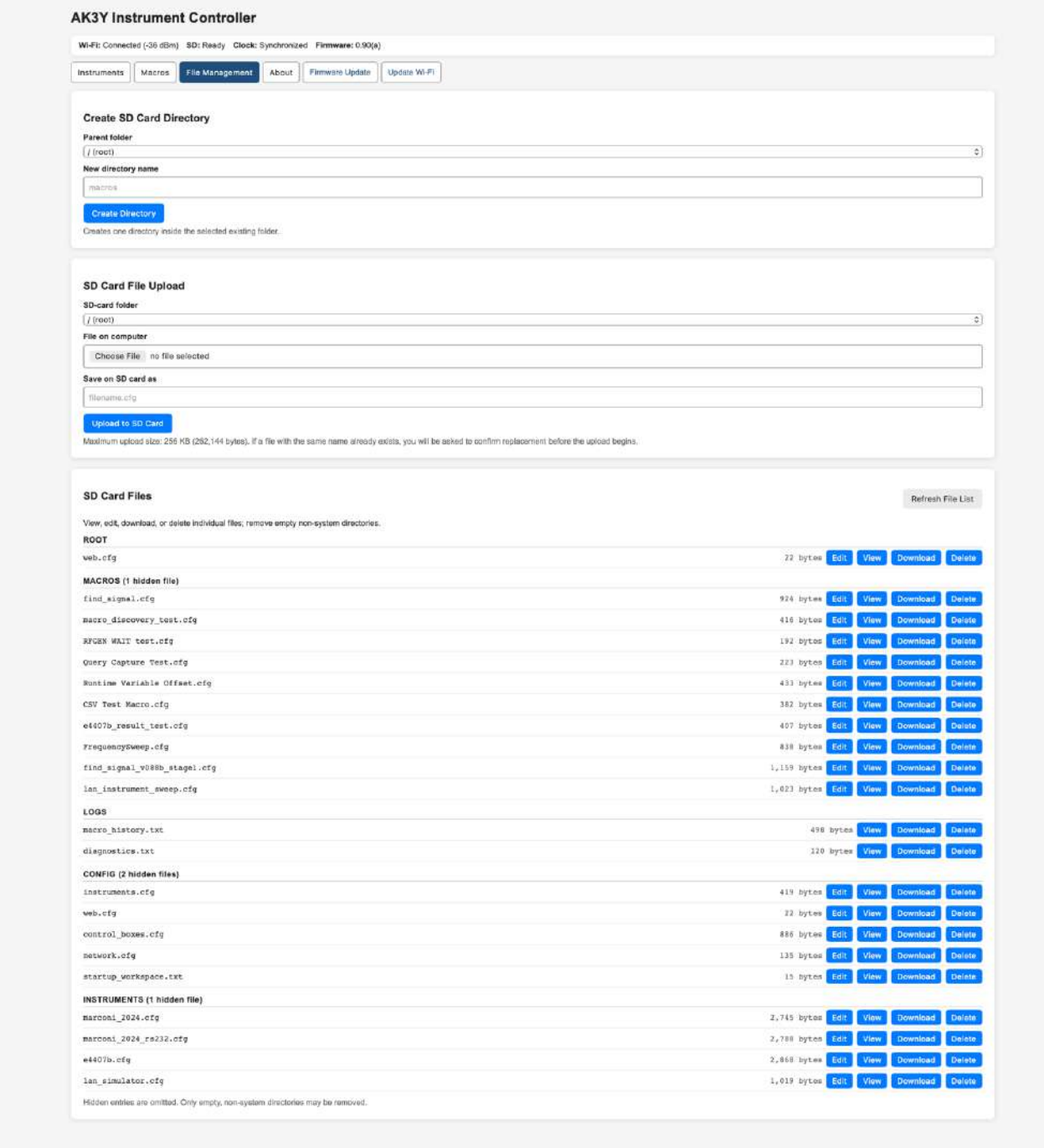


Figure 2-3. File Management screen showing microSD card directories and file operations.

The File Management screen provides browser-based access to the controller's microSD card. It contains three principal sections:

Create SD Card Directory allows the operator to create a new directory within a selected parent directory.

SD Card File Upload transfers a file from the user's computer to the controller's microSD card. The destination directory and stored filename can be specified. The interface supports uploads up to 256 KB and requests confirmation before replacing an existing file.

SD Card Files displays the files organized by directory. Depending on the file type, the available operations include Edit, View, Download, and Delete. Empty, non-system directories can also be removed.

The screenshot shows several important directories:

Directory	Purpose
/macros	Stores macro definitions
/logs	Contains diagnostic and macro-history files
/config	Stores controller configuration files
/instruments	Stores instrument profiles

This interface permits the operator to maintain instrument definitions, edit macros, retrieve measurement-related files, and examine diagnostic logs without physically removing the microSD card.

2.4 About Screen

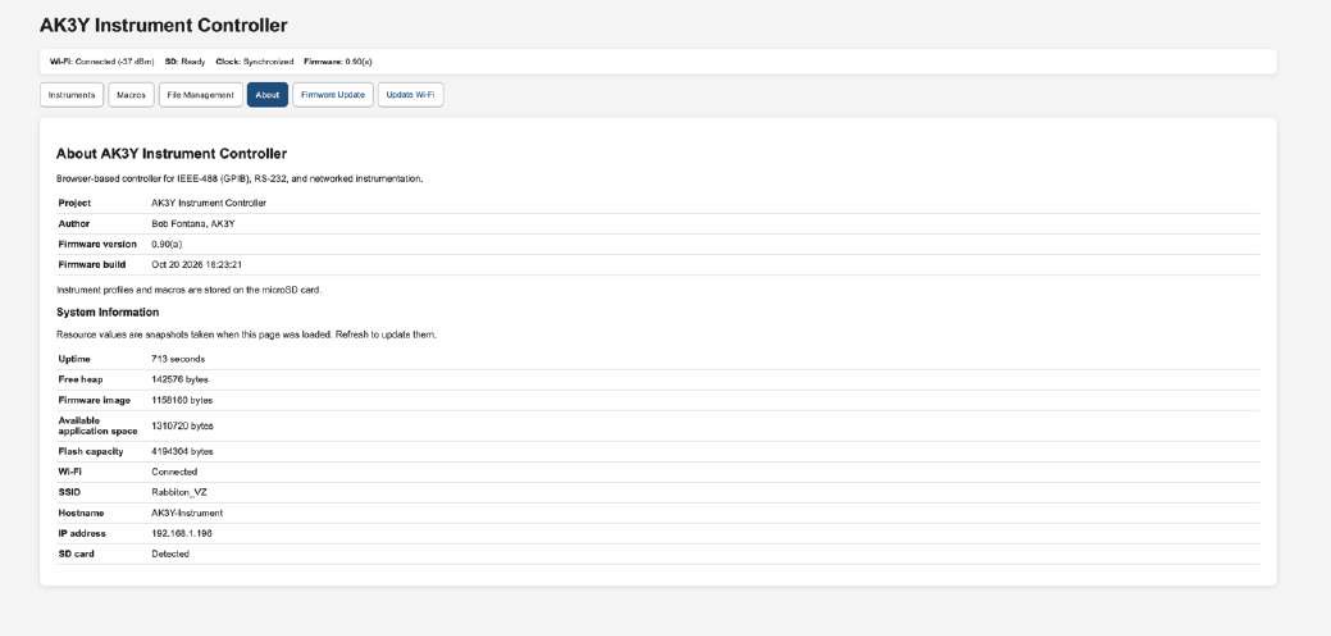


Figure 2-4. About screen showing firmware identification and controller system information.

The About screen provides identification and operating information for the controller.

The upper section identifies the project, author, firmware version, and firmware build date and time.

The System Information section reports operating parameters, including controller uptime, available heap memory, firmware image size, available application storage space, and total flash capacity.

Network information includes Wi-Fi connection status, network name (SSID), hostname, and assigned IP address. The screen also reports whether the microSD card has been detected.

These values are snapshots taken when the page is loaded. Refreshing the page obtains updated information.

The About screen is particularly useful for confirming the installed firmware version, identifying the controller's network address, and troubleshooting initialization or resource problems.

2.5 Firmware Update Screen

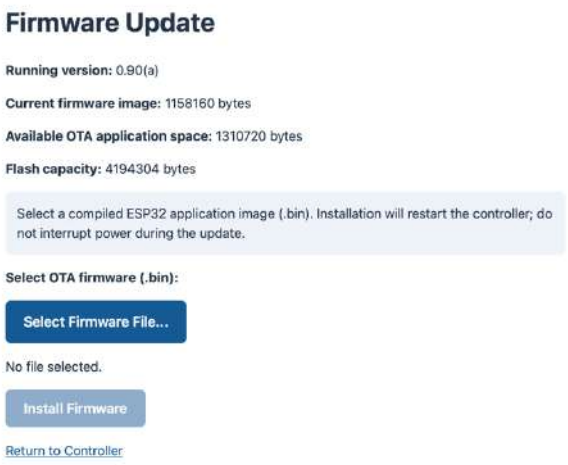


Figure 2-5. Firmware Update screen showing the browser-based firmware installation interface.

The Firmware Update screen provides a method of installing new controller firmware through the web browser, without requiring a direct USB connection to the controller.

The screen displays the currently installed firmware version, firmware image size, available over-the-air (OTA) application space, and total flash capacity.

To install an update, the operator selects a compiled ESP32 application image (.bin) from the local computer and initiates the installation.

The controller restarts as part of the firmware installation process. Power must not be interrupted while the update is in progress.

This capability is particularly convenient when the controller is installed in an equipment rack or another location where direct USB access is inconvenient.

2.6 Update Wi-Fi Screen

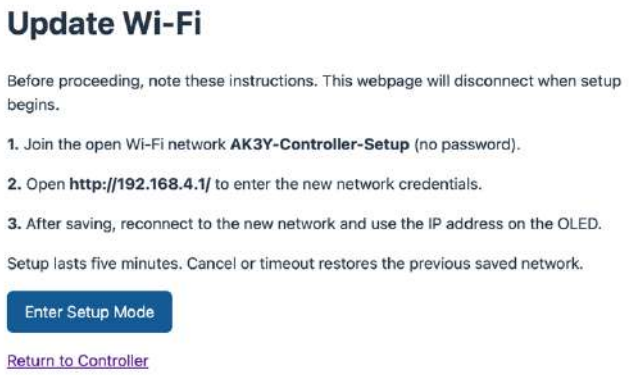


Figure 2-6. Update Wi-Fi screen showing the wireless network reconfiguration procedure.

The Update Wi-Fi screen allows the controller to be connected to a different wireless network.

Selecting Enter Setup Mode starts a temporary wireless configuration session. During this process, the existing browser connection is interrupted.

The operator connects a computer or mobile device to the controller's temporary wireless access point, AK3Y-Controller-Setup, and opens http://192.168.4.1/ to enter the new network credentials.

After the credentials have been saved, the operator reconnects to the new wireless network and accesses the controller using the IP address displayed on its OLED. The configuration session lasts up to five minutes. If setup is canceled or times out, the controller restores the previously saved network configuration.

This feature permits network changes without recompiling firmware or connecting the controller to a development computer.

3. Software Installation and Initial Startup

3.1 Introduction

The AK3Y Instrument Controller firmware runs on an Espressif ESP32-DevKitC-32E development board containing an ESP32-WROOM-32E module. The firmware provides the instrument communication interfaces, microSD card file system, Wi-Fi networking, embedded web server, and automated measurement facilities described in this manual.

There are two methods of installing the firmware:

1. Build and upload the firmware from source code using Visual Studio Code and the PlatformIO development environment.

2. Install precompiled firmware using a supplied binary image and an ESP32-compatible programming utility.

The first method is recommended for users who intend to examine or modify the source code. The second is more convenient for users who simply wish to operate the controller. Both methods require an initial USB connection between the development computer and the ESP32 controller.

Once the firmware has been installed and the controller has been configured for Wi-Fi operation, subsequent firmware updates can be performed through the web interface, as described in a later chapter.

3.2 Obtaining the Software

The firmware source code, configuration files, and installation resources are distributed through the project's GitHub repository.

****GitHub repository:** [Repository URL to be supplied]**

The repository contains the complete PlatformIO project, including the application source code, header files, build configuration, and all required supporting files. Precompiled firmware images are also supplied for users who do not wish to install a software development environment.

Instrument profiles, macro definitions, and other runtime configuration files are stored on the controller's microSD card. These files are separate from the firmware and must be installed or created as appropriate (see sections below).

3.3 Building and Installing the Firmware Using PlatformIO

3.3.1 Development Environment

The AK3Y Instrument Controller firmware was developed using Visual Studio Code and the PlatformIO development environment. The firmware uses the Arduino framework and is compiled using the GNU C++17 language standard.

Two external libraries are required:

Adafruit GFX Library

Adafruit SSD1306

Both libraries are specified in the project's platformio.ini configuration file and are installed automatically by PlatformIO.

3.3.2 Installing PlatformIO

Install Visual Studio Code from the official Microsoft website (https://code.visualstudio.com).

Within VS Code, open the Extensions panel, search for PlatformIO IDE, and install the extension. PlatformIO will install the necessary development tools, including the ESP32 compiler and Arduino framework, when the project is first built.

3.3.3 Obtaining the Source Code

Download or clone the AK3Y Instrument Controller GitHub repository (see URL in Section 3.2 above). Open the project directory in Visual Studio Code using File → Open Folder.

The selected directory must contain platformio.ini, which identifies the target board, compiler settings, and required libraries.

3.3.4 Compiling the Firmware

Select PlatformIO: Build from the VS Code command palette, or click the Build icon in the PlatformIO toolbar. PlatformIO will download any missing dependencies, compile the source files, and link the application.

The resulting firmware image is stored at

.pio/build/esp32dev/firmware.bin

A successful compilation is indicated by a SUCCESS message in the PlatformIO terminal.

3.3.5 Programming the ESP32

Connect the controller to the development computer using a USB data cable. Select PlatformIO: Upload. The project's default upload speed is 921600 baud. PlatformIO uses the ESP32 serial bootloader to transfer the firmware to the microcontroller's flash memory.

If programming fails because of a serial-port communication problem, close any active serial monitors, disconnect and reconnect the USB cable, and retry. If necessary, the upload speed can be reduced by changing the following entry in platformio.ini; e.g.:

upload_speed = 115200

Following a successful upload, the controller restarts and begins executing the newly installed firmware.

3.3.6 Monitoring Serial Output

PlatformIO provides a serial monitor for observing diagnostic messages generated by the controller. The serial monitor operates at 115200 baud, as specified by: monitor_speed = 115200.

To view the messages, select PlatformIO: Serial Monitor.

Serial output is particularly useful during initial installation, network configuration, and troubleshooting.

3.4 Installing Precompiled Firmware

Users who do not intend to modify the firmware can install a precompiled release without building the source code.

For initial installation on an unprogrammed ESP32, make sure that you select the release package which includes the required bootloader, partition table, application image, and any other flash components specified by the project's PlatformIO configuration. This will be identified in the GitHub repository.

The images must be programmed at the correct flash-memory addresses using an ESP32-compatible programming utility, such as Espressif's esptool.

The precise programming commands and flash addresses are supplied with the release package. These values must correspond to the partition layout and build configuration used to generate the firmware. (Note: A firmware image intended for an over-the-air update should not be assumed to be a complete initial-programming image.)

3.5 Preparing the microSD Card

The AK3Y Instrument Controller uses a microSD card to store instrument profiles, macro definitions, workspace configurations, and diagnostic logs.

Before installing the card, format it using the FAT32 file system. A microSD card with a capacity of 8 GB or 16 GB is recommended. Cards formatted using exFAT should not be used.

The controller organizes its files into several directories:

Directory	Contents
/instruments	Instrument profiles
/macros	Macro definitions
/config	Controller and workspace configuration files
/logs	Diagnostic and macro execution logs

On startup, the firmware automatically creates /config, /instruments, /macros, and /logs if they are absent. These four system directories are protected against deletion through the web interface. No files need to be copied to a freshly formatted FAT32 card before the controller is started.

The controller can start with no instrument definitions, instrument profiles, or macros installed. Instrument definitions and profiles are added after Wi-Fi setup, as described in Section 3.8. Macro files can be installed later when automated measurements are required.

3.6 Initial Startup and Wi-Fi Configuration

After installing the firmware and preparing the microSD card, apply power to the AK3Y Instrument Controller. During startup, the controller initializes its hardware interfaces, mounts the microSD card, and attempts to establish a Wi-Fi connection.

If valid Wi-Fi credentials have previously been saved in the ESP32's nonvolatile memory, the controller attempts to connect automatically.

If no valid credentials are available, or if the connection cannot be established, the controller enters Wi-Fi setup mode.

To configure the network:

1. Using a computer, tablet, or smartphone, connect to the wireless network named AK3Y-Controller-Setup. No password is required.

2. Open a web browser and navigate to http://192.168.4.1/.

3. Select the desired Wi-Fi network from the list of detected networks, or enter its name manually.

4. Enter the network password and select Test and save network.

5. The controller tests the connection before saving the credentials.

6. If the connection succeeds, the controller stores the credentials in nonvolatile memory and restarts automatically.

After restarting, the controller connects to the selected Wi-Fi network. Its assigned IP address is displayed on the OLED screen. Enter this IP address in a browser on a computer connected to the same network to access the controller's web interface.

* Note: Initial Wi-Fi setup has no time limit. The five-minute timeout described in Chapter 2 applies only when the user initiates a Wi-Fi configuration change from the normal web interface. Wi-Fi credentials are stored in the ESP32's nonvolatile memory rather than on the microSD card.

3.7 Verifying the Installation

Before connecting and operating laboratory instruments, verify that the controller has started correctly.

Confirm that:

The web interface is accessible from the browser.

The displayed firmware version corresponds to the installed release.

The microSD card is recognized.

The Wi-Fi connection is established.

The controller's About screen displays the expected system information.

The File Management screen displays the four system directories, even on a new card with no instrument profiles or macros. The /logs directory normally contains a diagnostics.txt file after startup.

Successful completion of these checks establishes that the controller firmware, network interface, and microSD storage are functioning. The absence of instrument profiles or macro files is normal at this stage.

3.8 Installing Instrument Definitions and Profiles

Instruments are configured by two types of files. The file /config/instruments.cfg lists the installed instruments, their interface types and connection settings, and the profile file associated with each instrument. Individual instrument profiles, stored as .cfg files in /instruments, define the functions, commands, and parameters presented in the Control Panels.

After the controller is connected to Wi-Fi, open its web interface and select File Management. In the SD Card File Upload section, choose /config as the destination and upload instruments.cfg. Next, select /instruments and upload each profile referenced by instruments.cfg. Verify the destination directory before each upload; the file list can be refreshed to confirm the files are in the correct locations.

For example, an instruments.cfg entry may identify a Marconi 2024 signal generator as a GPIB instrument at address 20 and refer to the profile marconi_2024.cfg. The definition belongs in /config/instruments.cfg, while the referenced profile belongs in /instruments/marconi_2024.cfg.

After uploading the definitions and profiles, restart the controller so that it reads the new instrument configuration. Reopen the Instruments screen, add a Control Panel, and select an installed instrument and function. If no instruments have been defined, or a referenced profile is missing, the interface provides guidance about the required files and directories.

Installing a profile makes its controls available in the browser; it does not, by itself, establish communication with a physical instrument. Before issuing commands, confirm that the instrument is connected, powered on, and configured for the specified interface and address or network settings.

3.9 Saving an Initial Workspace

Once a Control Panel has been configured, the operator may preserve its instrument selection, function, and displayed parameter values. In the Workspace Configuration section, select Save As..., enter a workspace name, and save the configuration. The controller creates /config/workspaces automatically when the first workspace is saved.

Select a saved workspace and check Load this configuration when the controller opens to designate it as the startup workspace. The controller records this selection in /config/startup_workspace.txt. Saved workspace configurations reside on the microSD card, not in the browser; they therefore accompany the card when it is moved to another controller.

A workspace can contain instrument Control Panels without any Macro Panels. Loading a workspace restores the displayed configuration but does not send commands to connected instruments. Save Configuration updates the currently loaded, named workspace; use Save As...

to create a new one. If a newly selected configuration has not yet been loaded, select Load before editing and saving it.

The following chapters describe the configuration and operation of individual instruments and automated measurement procedures.

4. Configuring and Operating Instruments

4.1 Introduction

The Instruments screen is the principal interface for directly controlling laboratory instruments connected to the AK3Y Instrument Controller. It provides graphical Control Panels through which the operator can adjust instrument settings, read measured values, select operating functions, and execute instrument-specific commands.

The controller supports instruments connected through IEEE-488 (GPIB), RS-232, and Ethernet (TCP) interfaces. Although these interfaces differ electrically and in their communications protocols, the browser presents a common operating environment for all three.

Each Control Panel is generated from an instrument profile stored on the controller's microSD card. The profile specifies which controls appear in the browser and the commands used to communicate with the instrument. Consequently, the controls presented for a signal generator may differ considerably from those for a spectrum analyzer, even though both are operated through the same web interface.

This chapter describes the normal procedures for configuring and operating instruments. Appendix A provides the reference for defining instruments and their communications interfaces, while Appendix B explains how to construct and modify instrument profiles.

4.2 Instrument Definitions and Profiles

Before an instrument can be selected in the browser, the controller must have two pieces of information: how to communicate with the instrument and which commands the instrument recognizes. These are supplied by two separate configuration files.

The first is the instrument inventory file, `/config/instruments.cfg`. This file identifies the available instruments, assigns each a descriptive name, and specifies its communications interface and associated settings.

For example, the following entry defines a Marconi 2024 RF signal generator connected to the GPIB bus at address 20:

```
[RFGEN]
name=RF Generator - GPIB
profile=marconi_2024.cfg
interface=GPIB
address=20
```

The second file, `/instruments/marconi_2024.cfg`, is the instrument profile. It contains the commands used to set and read the generator's frequency and RF output level, configure frequency sweeps, and perform other supported operations.

The distinction is important. The instrument inventory determines which instrument is selected and how it is reached, while the instrument profile determines which controls are displayed and which commands are sent.

Multiple entries in the inventory may reference the same profile. For example, a Marconi 2024 connected through GPIB and another connected through RS-232 can share a profile when their command languages are compatible. Each inventory entry supplies its own interface and communications settings.

The installation and upload procedures are described in Section 3.8. After modifying the instrument inventory, restart the controller so that the updated definitions are loaded.

4.3 Creating an Instrument Control Panel

After the controller has started and established its Wi-Fi connection, open a browser on a computer connected to the same network. Enter the controller's IP address, which is displayed on its OLED screen.

Select **Instruments** from the navigation menu.

The Instruments screen contains the area in which instrument Control Panels are displayed. A new installation may initially have no panels, even when instrument profiles have already been installed.

To create a Control Panel, select **+ Add Control Panel**. A new panel appears with an Instrument selection menu.

Open this menu and select the desired instrument. The available choices are obtained from the instrument inventory file. For example, selecting RF Generator - GPIB creates a Control Panel for the Marconi 2024 signal generator.

Once an instrument is selected, the panel displays its identification and connection information, together with the controls defined in its profile.

For our Marconi example, the panel identifies:

Instrument: RF Generator - GPIB
Manufacturer and model: Marconi 2024
Interface: GPIB
Address: 20

The instrument's available operating functions are presented through the Function selector.

If the instrument inventory contains no valid entries, there will be no instruments available for selection. Similarly, if a referenced profile is missing or invalid, the expected controls may not be available. The appropriate configuration files should be checked before proceeding.



Figure 4-1. Marconi 2024 Control Panel showing the Basic RF Generator function, with Frequency and RF Level controls.

4.4 Understanding the Control Panel

Each Control Panel contains several common elements, regardless of the type of instrument being controlled.

At the top of the panel is its identifying number, such as Control Panel 1. A **Remove** button allows the panel to be deleted from the current workspace.

Immediately below is the Instrument selector. This determines which configured instrument the panel represents.

The main instrument area displays the selected instrument's name, manufacturer, model, and communications interface.

Below this information is the Function selector. It determines which group of instrument controls is currently displayed.

The remaining controls depend on the selected instrument and function. They may include numerical entry fields, selection menus, command buttons, and controls whose visibility changes according to the instrument's operating mode.

For the Marconi 2024, selecting **Basic RF Generator** displays two numerical controls: Frequency and RF Level. Each has an entry field and separate Set and Read buttons.

The frequency control also displays the permissible operating range defined in the profile:

```
Profile range: 9000 to 2400000000 Hz
```

The RF Level control similarly displays its permissible range of −140 to +13 dBm.

The panel also includes READ and SEND buttons below the instrument-specific controls. These provide panel-level operations distinct from the individual Set and Read buttons.

The precise controls displayed are determined by the selected instrument profile. They are not permanently programmed into the browser interface.

4.5 Setting and Reading Instrument Parameters

4.5.1 Setting a Numerical Value

To change an instrument parameter, enter the desired value in its corresponding field and select **Set**.

For example, to set the Marconi 2024 output frequency to 796.009 MHz, enter:

```
796.009 MHz
```

in the Frequency field and select Set.

The controller interprets the engineering units and converts the entered value into the base units specified by the instrument profile.

For the Marconi, the resulting instrument command is:

```
CFRQ:VALUE 796009000
```

The controller transmits this command using the interface specified in the instrument inventory.

The operator does not need to know the underlying command syntax during normal operation. The appropriate command is supplied by the instrument profile.

The same procedure applies to the RF output level. Entering:

```
-20 dBm
```

and selecting Set causes the controller to send:

```
RFLV:VALUE -20DBM
```

The controller checks numerical entries against the limits specified in the profile before transmitting Set commands.

4.5.2 Reading a Parameter

The Read button retrieves the current value of a parameter from the instrument.

For example, selecting Read beside Frequency causes the controller to send the Marconi command:

```
CFRQ?
```

The instrument returns its frequency information. The controller extracts the numerical value according to the response-processing instructions in the profile and displays the result in the corresponding field.

This distinction between Set and Read is useful because the displayed value may not always represent the instrument's actual state.

For example, the instrument may have been adjusted manually from its front panel, or changed by another controller command. Selecting Read obtains the current value reported by the instrument rather than relying solely on the value previously entered in the browser.

4.5.3 Engineering Units

The controller supports engineering units appropriate to the physical quantity being controlled.

For frequency, the operator may enter values in Hz, kHz, MHz, or GHz. Thus, the following entries represent the same frequency:

```
796009000 Hz
796009 kHz
796.009 MHz
0.796009 GHz
```

The controller converts these entries into the base units specified in the profile.

Other supported quantities include time, voltage, current, power, attenuation, resistance, and percentage. The complete list of recognized quantities and permissible units is provided in Table B-3 of Appendix B.

The supported units depend on the quantity. For example, an RF power level defined using `quantity=power_level` is expressed in dBm; the controller does not automatically convert this quantity between dBm and watts.

4.5.4 Reading Versus Setting

An important distinction should be made between the value displayed in a Control Panel and the actual operating condition of the instrument. Entering a new value does not necessarily change the instrument until Set is selected.

Likewise, restoring a saved workspace may populate the entry fields with previously saved values without transmitting those values to the instrument.

When the actual instrument settings are important, use the Read controls to retrieve the current values before proceeding. This is particularly advisable when an instrument has been adjusted from its front panel or used in an automated measurement sequence.

4.6 Selecting Instrument Functions

An instrument may provide several distinct operating functions. Rather than displaying every possible control simultaneously, the controller organizes related controls through the Function selector.

The Marconi 2024 provides a useful example. Its profile defines two functions:

Basic RF Generator provides ordinary fixed-frequency operation, with controls for frequency and RF output level.

Carrier Frequency Sweep provides the controls required to configure and execute frequency sweeps.

The name Basic RF Generator is specified by the `default_function` parameter in the Marconi profile. It is simply the name assigned to the instrument's basic function.

The additional Carrier Frequency Sweep function is defined by a `type=group` section, as described in Appendix B.

4.6.1 Selecting Basic RF Generator

When Basic RF Generator is selected, the panel displays the Frequency and RF Level controls.

The Marconi profile also defines an entry command:

```
CFRQ:MODE FIXED
```

This command places the generator in fixed-frequency mode when the basic function is entered.

The entry command is specific to the instrument. Other profiles may use different commands or may not require a command when changing functions.

4.6.2 Selecting Carrier Frequency Sweep

Selecting Carrier Frequency Sweep changes the set of controls displayed in the panel.

The sweep function provides settings for starting and stopping frequencies, step size, step time, sweep mode, sweep type, and triggering.

The profile also defines:

```
enter=CFRQ:MODE SWEPT
```

This command places the Marconi in swept-frequency mode when the function is entered.

The available sweep controls include numerical fields and selection menus. For example, Sweep Mode offers Single and Continuous operation, while Sweep Type offers Linear and Logarithmic sweeping.

The selected sweep type also determines which step-size control is displayed. For a linear sweep, the controller displays Step Size, expressed as a frequency increment. For a logarithmic sweep, it displays Log Step, expressed as a percentage.

This behavior is defined in the instrument profile using the `visible_when` parameter.

The sweep function also provides action buttons for starting, stopping, and resetting a sweep.

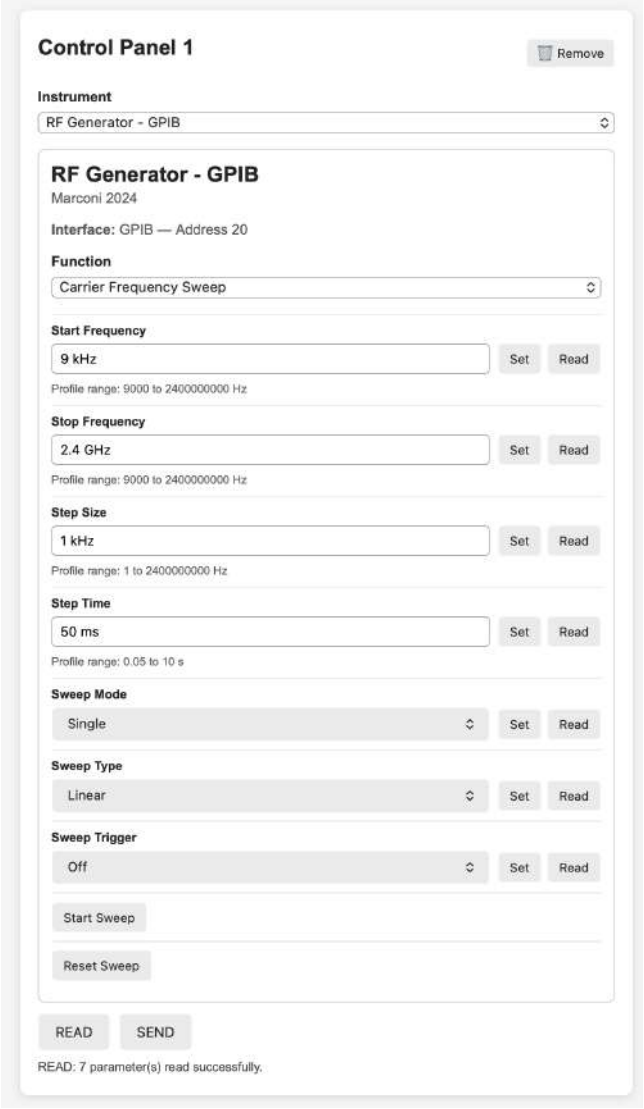


Figure 4-2. Marconi 2024 Control Panel showing the Carrier Frequency Sweep function.

4.6.3 Action Buttons

Some instrument operations do not require a numerical value or a selection.

For example, the Marconi's Reset Sweep button sends:

```
SWEEP:RESET
```

Other actions may use a two-state button. In the Marconi profile, the Start Sweep button changes to Stop Sweep after it has been activated.

The first operation sends `SWEEP:GO`, while the second sends `SWEEP:HALT`.

These buttons provide convenient access to instrument commands without requiring the operator to enter the commands manually.

The button's displayed state should not necessarily be interpreted as confirmation of the instrument's actual state, particularly if the instrument can also be controlled independently.

4.7 Operating Multiple Instruments

One of the controller's principal advantages is its ability to operate several instruments through a common browser interface.

The Instruments screen supports multiple independent Control Panels. Each panel can be assigned to a different instrument and can display its own selected function.

For example, Control Panel 1 might operate the Marconi 2024 signal generator, while Control Panel 2 operates an Agilent E4407B spectrum analyzer.

The Marconi panel could be used to set the output frequency and RF level, while the E4407B panel could be used to adjust the analyzer's frequency range, resolution bandwidth, and other supported settings.

The two panels may use different communications interfaces. One instrument could be connected through GPIB, another through RS-232, and a third through TCP. The controller handles the interface-specific communication according to each instrument's inventory entry.

Additional Control Panels can be created using + Add Control Panel. Individual panels can be removed using their Remove buttons.

Removing a panel changes the browser workspace; it does not remove the corresponding instrument definition or profile from the microSD card.

Multiple panels are particularly useful when configuring a measurement involving both a signal source and a measurement instrument.

For repetitive or coordinated operations involving several instruments, macros provide a more powerful approach. These are discussed in a subsequent chapter.

4.8 Using the Advanced Command Console

The Instruments screen also includes an Advanced Command Console, located below the Control Panels and workspace controls.

This facility provides a way to execute controller commands directly, without relying exclusively on the graphical controls generated from instrument profiles.

It is particularly useful for testing instrument communications, experimenting with instrument commands, and performing operations that may not have a corresponding graphical control.

Unlike an instrument-specific Set button, which automatically constructs a command from its profile, the Advanced Command Console requires the operator to enter a command using the controller's supported command syntax.

The console is therefore intended primarily for users who are familiar with the controller's command language and the instruments being operated.

A complete description of the supported console commands, arguments, and examples can be found in Appendix D.

4.9 Saving and Restoring Control Panel Arrangements

After configuring one or more Control Panels, the operator may wish to preserve the arrangement for later use. The Workspace Configuration section of the Instruments screen provides facilities for saving and restoring these arrangements.

A saved workspace can preserve the Control Panels, their selected instruments, their selected functions, and displayed parameter values. It may also include Macro Panels.

To create a new saved workspace, select **Save As...**, provide a descriptive name, and save the configuration.

Previously saved workspaces can be selected and loaded through the workspace controls.

The controller also permits one saved workspace to be designated as the startup configuration. When this option is enabled, the selected workspace is restored when the controller interface is opened.

Workspace files are stored in the `/config/workspaces` directory on the microSD card.

Note: Loading a workspace does not automatically send its saved parameter values to the connected instruments. It restores the browser configuration, not necessarily the physical operating conditions of the equipment.

For example, a workspace may display a Marconi frequency of 146 MHz because that value was saved previously. The generator itself may currently be set to a different frequency.

Use Read to obtain the actual instrument setting, or Set to apply the displayed value deliberately.

This separation between saved browser settings and physical instrument settings helps avoid unintended changes when loading a workspace.

4.10 Communication Errors and Troubleshooting

An instrument may appear correctly in a Control Panel even when the controller cannot communicate with it.

This is because the Control Panel is generated from the stored inventory and profile files. Its appearance does not, by itself, confirm that the physical instrument is connected or responding.

If a Set or Read operation fails, first confirm that the instrument is powered on and configured for remote operation through the intended interface.

For GPIB instruments, verify the cable connection and confirm that the address configured in `/config/instruments.cfg` matches the address set on the instrument.

For RS-232 instruments, check the selected controller port, baud rate, data bits, parity, and stop bits. The instrument and controller must use compatible settings.

For TCP instruments, confirm the configured host address and port, and verify that the instrument is accessible on the network.

A communication failure can also result from an incorrect command or an unexpected response format in the instrument profile.

For example, a Set command may be accepted by the instrument while a Read operation fails because the profile specifies an incorrect response prefix or terminator.

When troubleshooting, it is useful to distinguish among three possible problems:

> **Instrument selection or configuration:** The instrument does not appear in the selection menu, or its expected controls are missing.
>
> **Communications:** The instrument is displayed correctly, but commands cannot be exchanged.
>
> **Command interpretation:** Communication succeeds, but an instrument rejects a command or the controller cannot interpret its response.

The first category generally involves the inventory or profile files. The second involves physical connections and interface settings. The third usually requires checking the instrument programming manual and the corresponding profile definitions.

The controller's diagnostic logs and Advanced Command Console may provide additional information when investigating these problems.

For persistent difficulties, begin with a simple command or query known to be supported by the instrument, establish reliable communication, and then test more complex operations.

4.11 Summary

The Instruments screen provides a common operating environment for GPIB, RS-232, and TCP instruments.

Each Control Panel is created from an instrument profile, allowing controls to be tailored to the capabilities of individual instruments without modifying the controller firmware.

The operator can create and remove Control Panels, select instrument functions, set and read parameters, execute actions, and operate multiple instruments from the same browser window. Control Panel arrangements can also be saved as workspaces and restored for later use.

Together, these facilities provide the foundation for the more advanced automated measurement operations described in the following chapters.

5. Creating and Executing Macros

5.1 Introduction

The AK3Y Instrument Controller provides a macro facility for automating measurements and other operations involving one or more instruments. A macro is a sequence of commands executed by the controller, eliminating the need for the operator to perform each operation manually through an instrument Control Panel.

Macros are particularly useful for repetitive measurements, coordinating several instruments, recording measurement results, and carrying out procedures that would otherwise require numerous manual operations.

For example, a macro might command an RF signal generator to a particular frequency, initiate a sweep on a spectrum analyzer, read the frequency and amplitude of a measured signal, and record the results in a file. This sequence can then be repeated over a range of frequencies without further operator intervention.

The controller's macro facility uses instrument capabilities already defined in the instrument profiles described in Appendix B. Thus, a macro does not ordinarily need to contain the actual GPIB, RS-232, or TCP command strings recognized by an instrument. Instead, it refers to the instrument and the desired capability, allowing the controller to construct and transmit the appropriate command.

This chapter explains how macros are organized, selected, and executed, using two working macro examples: **FrequencySweep** and **FindSignal**. The complete macro command language is documented in Appendix C.

5.2 Macro Files

Macros are stored as plain-text files with the extension `.cfg` in the `/macros` directory of the controller's microSD card.

Each macro consists of sections enclosed in square brackets, followed by parameter definitions or numbered instructions.

A typical macro contains three principal elements:

1. A `[macro]` section identifying the purpose of the macro.
2. One or more `[parameter:...]` sections defining values that the operator may enter before execution.
3. A `[steps]` section containing the commands to be executed.

Macros may also contain `[result:...]` sections that define measurement results to be displayed after execution.

The name displayed in the browser's Macro selector is derived from the filename. For example, `FrequencySweep.cfg` appears as **FrequencySweep**, while `FindSignal.cfg` appears as **FindSignal**.

The optional `name=` parameter within the `[macro]` section does not determine the displayed macro name.

Macro files can be created or modified using an ordinary plain-text editor (or via the controller's built-in text editing facility) and uploaded through the controller's File Management screen.

5.3 Selecting a Macro

Macros are accessed from the **Macros** screen of the controller's browser interface. The operator can create a Macro Panel and select one of the available macro files. Once a macro has been selected, the controller displays its description and any input parameters defined in the file.

The parameter fields allow the operator to supply the values needed for the measurement without editing the macro itself.

For example, the FrequencySweep macro provides three inputs:

- Start Frequency
- Stop Frequency
- Frequency Increment

Each field initially contains the default value specified in the macro file. The operator may change these values before executing the macro. A macro can therefore be reused for different measurement conditions simply by changing its input parameters.

5.4 Example 1: FrequencySweep

Our first example uses two instruments: the Marconi 2024 RF signal generator and the Agilent E4407B 26.5 GHz spectrum analyzer.

The purpose of the macro is to step the Marconi through a specified frequency range, measure the resulting signal with the spectrum analyzer, and record the measurements in a CSV file.

The macro is stored as `FrequencySweep.cfg`.

5.4.1 Macro Description

The file begins with:

```
[macro]
description=Sweep the Marconi 2024 and measure each point with the E4407B
```

The `[macro]` section provides descriptive information about the macro. The `description` parameter explains its purpose and is intended to help the operator understand what the macro will do.

5.4.2 Input Parameters

The next three sections define the values entered by the operator before execution.

```
[parameter:start_frequency]
type=number
quantity=frequency
label=Start Frequency
base_unit=Hz
default=147MHz

[parameter:stop_frequency]
type=number
quantity=frequency
label=Stop Frequency
base_unit=Hz
default=149MHz

[parameter:increment]
type=number
quantity=frequency
label=Frequency Increment
base_unit=Hz
default=100kHz
```

Each section begins with `[parameter:name]`, where `name` is the internal identifier used to reference the parameter within the macro. For example, `[parameter:start_frequency]` defines the parameter named `start_frequency`.

The `type=number` parameter specifies a numerical input. The `quantity=frequency` and `base_unit=Hz` parameters tell the controller that the value represents a frequency, allowing the operator to use recognized engineering units such as kHz and MHz.

The `label` parameter specifies the text displayed in the Macro Panel.

Finally, the `default` parameter supplies the initial value displayed in the field.

In this example, the defaults establish a frequency range from 147 MHz to 149 MHz in increments of 100 kHz. Note that these values are not fixed. They may be changed in the Macro Panel prior to execution.

5.4.3 Macro Instructions

The remainder of the file contains the actual measurement sequence:

```
[steps]
1=SWEEP Freq_IN {start_frequency} {stop_frequency} {increment}
2=SET RFGEN frequency {Freq_IN}
3=WAIT RFGEN ready
4=ACTION SPECTRUM single_sweep
5=WAIT SPECTRUM ready
```

```
6=ACTION SPECTRUM peak_search
7=QUERY SPECTRUM marker_frequency AS Freq_Measured
8=QUERY SPECTRUM marker_amplitude AS Power_Measured
9=LOG /logs/frequency_sweep.csv Freq_IN,Freq_Measured,Power_Measured
10=ENDSWEEP
11=ACTION SPECTRUM continuous_sweep
```

The `[steps]` section contains numbered commands, executed in sequence.

The first command is:

```
SWEEP Freq_IN {start_frequency} {stop_frequency} {increment}
```

This begins a sweep loop using the three input parameters defined earlier.

The variable `Freq_IN` represents the current frequency within the loop. Its value begins at the `start_frequency` and changes by the specified `increment` (which can be positive or negative for decreasing values) as the loop proceeds. `SWEEP` stops when `Freq_IN` equals or exceeds `stop_frequency` (positive `increment`) or equals or is less than `stop_frequency` (negative `increment`). Think of this behavior as that of a typical “FOR” loop.

The expressions enclosed in braces refer to the input parameters. For example, `{start_frequency}` is replaced by the value entered in the Start Frequency field.

The second instruction is:

```
SET RFGEN frequency {Freq_IN}
```

This commands the instrument identified as `RFGEN` to set its `frequency` capability to the current value of `Freq_IN`. The identifier `RFGEN` must correspond to the instrument identifier configured in the controller's instrument inventory. The capability `frequency` must be defined in that instrument's profile.

The controller obtains the actual instrument command from the profile. For the Marconi 2024, the command is constructed using the frequency setting defined in Appendix B.

The next instruction is:

```
WAIT RFGEN ready
```

This performs the readiness check defined in the Marconi profile. It allows the macro to confirm that the signal generator has completed the requested operation before proceeding.

The fourth instruction is:

```
ACTION SPECTRUM single_sweep
```

This tells the spectrum analyzer to perform a single sweep. The command is obtained from the analyzer's instrument profile.

As before, the following instruction,

```
WAIT SPECTRUM ready
```

performs the instrument's readiness check (in this case, the spectrum analyzer) before the measurement continues.

Once the sweep has completed, the macro executes:

```
ACTION SPECTRUM peak_search
```

This invokes the analyzer's peak-search capability, locating the strongest signal in the current analyzer display.

The next two commands retrieve the measured frequency and amplitude:

```
QUERY SPECTRUM marker_frequency AS Freq_Measured
QUERY SPECTRUM marker_amplitude AS Power_Measured
```

The `QUERY` operation reads the specified instrument capability.

The keyword `AS` assigns the returned value to a macro variable. Here, the measured frequency is stored in `Freq_Measured`, while the measured amplitude is stored in `Power_Measured`.

The next instruction records the measurement:

```
LOG /logs/frequency_sweep.csv Freq_IN,Freq_Measured,Power_Measured
```

This writes the current input frequency and the two measured values to the specified CSV file on the microSD card.

The `ENDSWEEP` command marks the end of the sweep loop. The controller repeats the commands within the loop for the successive frequencies until the sweep is complete.

Finally, the macro executes:

```
ACTION SPECTRUM continuous_sweep
```

This returns the spectrum analyzer to continuous sweeping after the automated measurement has finished.

5.4.4 Measurement Results

The resulting CSV file contains three measurement quantities:

- The frequency commanded to the signal generator.
- The frequency measured by the spectrum analyzer.

- The amplitude measured by the spectrum analyzer.

The file can be downloaded through the controller's File Management screen and opened using spreadsheet or data-analysis software. For example, the data may be used to examine the measured output level over frequency, compare commanded and measured frequencies, or generate a plot of the measurement results.

The important feature of this example is that the entire procedure is performed automatically. The operator need only specify the frequency range and increment, then execute the macro.

5.5 Example 2: FindSignal

The second example, `FindSignal.cfg`, illustrates a somewhat different measurement technique. Instead of stepping a signal generator through a series of frequencies, this macro uses the Agilent E4407B spectrum analyzer to locate and identify the strongest signal within a specified frequency range. It progressively narrows the analyzer's frequency span around the strongest detected signal, then reports the final measured frequency and amplitude.

5.5.1 Input Parameters

The macro begins with:

```
[macro]
name=Find Signal
description=Find the strongest signal within a specified frequency range
```

As discussed earlier, the filename `FindSignal.cfg` determines the name displayed in the Macro selector.

The macro defines three operator inputs:

```
[parameter:start]
type=number
label=Start Frequency
quantity=frequency
base_unit=Hz
default=1MHz

[parameter:stop]
type=number
label=Stop Frequency
quantity=frequency
base_unit=Hz
default=200MHz

[parameter:span]
type=number
label=Final Span
quantity=frequency
base_unit=Hz
default=100kHz
```

The first two parameters define the initial frequency range to be searched. The third parameter, `Final Span`, specifies the narrow frequency span used for the final measurement.

With the default values, the macro searches from 1 MHz to 200 MHz and ultimately reduces the analyzer's span to 100 kHz.

5.5.2 Defining Displayed Results

Unlike FrequencySweep, which writes its measurements to a CSV file, FindSignal defines two results for display in the Macro Panel.

```
[result:PeakFrequency]
label=Peak Frequency
source=PeakFreq
unit=Hz

[result:PeakAmplitude]
label=Peak Amplitude
source=PeakLevel
unit=dBm
```

Each `[result:...]` section defines an output quantity. The `label` parameter specifies the text displayed to the operator.

The `source` parameter identifies the macro variable containing the measurement result.

The `unit` parameter specifies the unit associated with that result.

Thus, the result labeled `Peak Frequency` obtains its value from the macro variable `PeakFreq`, while `Peak Amplitude` obtains its value from `PeakLevel`.

These variables are populated by instrument queries near the end of the macro.

5.5.3 Locating the Strongest Signal

The macro begins by presetting the spectrum analyzer and establishing the initial search range:

```
[steps]
1=ACTION SPECTRUM preset
2=DELAY 2000
3=SET SPECTRUM start_frequency {start}
4=SET SPECTRUM stop_frequency {stop}
5=DELAY 2000
```

The first instruction invokes the analyzer's preset capability. The `DELAY` instruction pauses execution for a specified number of milliseconds. Thus, `DELAY 2000` introduces a two-second delay.

The following `SET` commands establish the start and stop frequencies supplied by the operator.

The macro then performs a peak search and moves the detected peak to the center of the analyzer display:

```
6=ACTION SPECTRUM peak_search
7=DELAY 2000
8=ACTION SPECTRUM marker_to_center
9=DELAY 2000
```

This sequence identifies the strongest signal in the initial frequency range and adjusts the analyzer's center frequency accordingly.

The next instructions narrow the analyzer's span to 2 MHz and repeat the operation:

```
10=SET SPECTRUM span 2000000
11=DELAY 2000
12=ACTION SPECTRUM peak_search
13=DELAY 2000
14=ACTION SPECTRUM marker_to_center
15=DELAY 2000
```

Finally, the macro applies the user-specified Final Span and performs another peak search:

```
16=SET SPECTRUM span {span}
17=DELAY 2000
18=ACTION SPECTRUM peak_search
19=DELAY 2000
20=ACTION SPECTRUM marker_to_center
21=DELAY 2000
22=ACTION SPECTRUM peak_search
23=QUERY SPECTRUM marker_frequency AS PeakFreq
24=QUERY SPECTRUM marker_amplitude AS PeakLevel
```

The final two queries retrieve the frequency and amplitude of the strongest signal.

These values are assigned to the macro variables `PeakFreq` and `PeakLevel`, which supply the results displayed in the Macro Panel.

5.5.4 Understanding the Measurement

FindSignal illustrates how a relatively short macro can perform a useful measurement procedure that would otherwise require numerous manual operations. The macro first searches a broad frequency range, centers the strongest detected signal, and then narrows the span in stages to improve the final frequency determination.

Unlike FrequencySweep, this macro does not contain a loop. It performs a fixed sequence of operations. It also uses fixed delays rather than the instrument readiness checks employed in FrequencySweep.

The two examples therefore demonstrate different approaches to coordinating instrument operations. FrequencySweep is organized around a repeated measurement loop and records its results in a CSV file. FindSignal executes a fixed sequence and presents its final measurements directly in the Macro Panel.

5.6 Macro Variables and Parameters

The two examples illustrate several ways that values are used within macros.

Input parameters are defined in `[parameter:...]` sections and entered by the operator before execution. They are referenced in macro instructions using braces, such as `{start}` or `{increment}`.

Loop variables are created by operations such as `SWEEP`. In FrequencySweep, `Freq_IN` represents the current frequency being measured.

Measurement variables are created when a `QUERY` instruction uses the `AS` keyword. For example:

```
QUERY SPECTRUM marker_amplitude AS PeakLevel
```

stores the returned amplitude in the variable `PeakLevel`.

These variables may then be used by subsequent macro instructions, written to log files, or associated with displayed results.

The distinction between input parameters and measurement variables is important. Input parameters specify what the operator wants the macro to do, while measurement variables hold information obtained during execution.

5.7 Executing and Monitoring a Macro

After selecting a macro in the Macros screen, review its description and input parameters.

Enter the desired values, ensuring that they are appropriate for the connected instruments and the intended measurement.

Before execution, confirm that all required instruments are powered on, connected, and configured for the correct communications interfaces.

The macro's instrument identifiers must correspond to entries in the controller's instrument inventory. For example, both examples use the identifier `SPECTRUM` to refer to the spectrum analyzer, while FrequencySweep also uses `RFGEN` for the signal generator.

Once execution begins, the controller processes the numbered instructions in sequence, communicating with the instruments as required.

Depending on the macro, results may appear in the Macro Panel or be recorded in files on the microSD card.

For long-running measurements, the execution time may depend on instrument response times, sweep durations, deliberate delays, and the number of measurements being performed.

A frequency sweep containing many points may therefore take considerably longer than a macro that executes only a few commands.

5.8 Macro Errors and Troubleshooting

A macro may fail even when its syntax is correct.

For example, an instrument may be disconnected, configured for the wrong interface, or unable to respond to a command.

An instrument identifier used in a macro may not match the corresponding identifier in the inventory file. Similarly, a capability referenced by `SET`, `QUERY`, or `ACTION` may not exist in the selected instrument profile.

Incorrect parameter values can also cause problems. For example, a requested frequency may exceed the instrument's permitted operating range.

Readiness checks and delays deserve particular attention.

A `WAIT` instruction relies on the readiness parameters defined in the instrument profile. If the instrument does not return the expected response within the specified timeout, the operation may fail. A `DELAY` instruction, by contrast, simply pauses execution for the specified interval. It does not verify that the instrument has completed its operation. For this reason, readiness checks are generally preferable when the instrument provides a suitable completion query.

When developing a new macro, begin with a small number of instructions and verify that each operation works correctly before adding loops, logging, or more complex measurement sequences.

The Advanced Command Console described in Chapter 4 may also be useful for testing individual instrument operations.

5.9 Summary

Macros extend the AK3Y Instrument Controller from manual instrument operation to automated measurement procedures.

A macro can set instrument parameters, execute actions, wait for instrument readiness, introduce delays, query measurement values, repeat operations, and record or display results.

The FrequencySweep example demonstrates coordinated operation of a signal generator and spectrum analyzer, including repeated measurements and CSV logging.

The FindSignal example demonstrates a sequential measurement procedure that progressively narrows a spectrum analyzer's frequency span and reports the strongest detected signal.

Both examples use the same underlying instrument profiles employed by the graphical Control Panels. This allows the operator to automate instruments without embedding their complete command languages in every macro.

Appendix C provides the detailed macro language reference, including the complete syntax of the supported instructions, parameters, and result definitions.

6. Workspace Management

6.1 Introduction

The AK3Y Instrument Controller allows the operator to create and save workspace configurations for later use. A workspace represents the arrangement of instrument Control Panels and Macro Panels displayed in the browser, together with their associated selections and operating parameters.

For example, a workspace might contain two Control Panels, one for a Marconi 2024 RF signal generator and another for an Agilent E4407B spectrum analyzer. A second workspace might contain the same instruments, but with different functions selected and a Macro Panel configured for automated measurements.

Rather than recreating these arrangements each time the controller is used, the operator can save them under descriptive names and restore them when needed. Workspace configurations are stored on the controller's microSD card, making them available whenever the browser interface is opened.

An important distinction must be emphasized: **a workspace preserves the browser configuration, not necessarily the actual operating state of the connected instruments.** Loading a workspace does not automatically transmit its saved parameter values to the instruments.

6.2 The Workspace Configuration Controls

The Workspace Configuration section is located on the Instruments screen, below the instrument Control Panels.

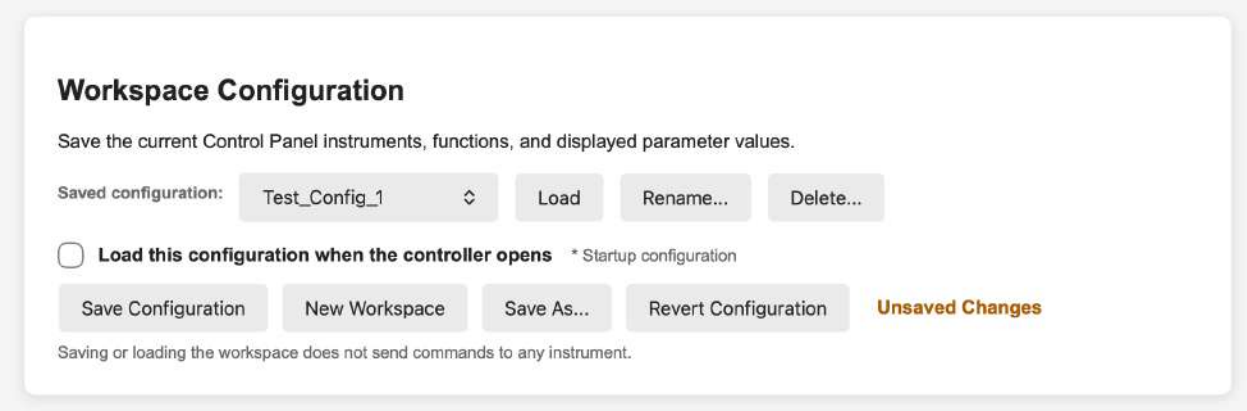


Figure 6-1. Workspace Configuration section of the Instruments screen.

The principal controls are:

Table 6-1. Workspace Configuration Controls

Control	Function
Load	Loads the selected saved workspace.
Rename...	Renames the selected saved workspace.
Delete...	Deletes the selected saved workspace.
Load this configuration when the controller opens	Designates the selected workspace as the startup configuration.
Save Configuration	Saves the current workspace configuration.
New Workspace	Starts a new workspace configuration.
Save As...	Saves the current configuration under a new name.
Revert Configuration	Restores the previously saved configuration, discarding unsaved changes.

The workspace designated for automatic startup is identified by an asterisk appended to its name in the workspace selector (see Section 6.7)

These controls allow the operator to maintain several independent configurations for different instruments or measurement procedures.

6.3 Creating a Workspace

A workspace can be created after one or more Control Panels or Macro Panels have been configured.

As an example, suppose that the operator wishes to create a workspace for measurements involving the Marconi 2024 signal generator and E4407B spectrum analyzer.

Begin by opening the Instruments screen and selecting **+ Add Control Panel**. Select the Marconi 2024 from the Instrument menu and choose the desired function, such as Basic RF Generator.

Add a second Control Panel and select the E4407B spectrum analyzer. Choose the analyzer function appropriate to the intended measurement.

The two Control Panels can now be used independently to adjust instrument settings and read measurements.

If desired, a Macro Panel can also be added on the Macros screen and configured for a particular measurement procedure.

Once the desired arrangement has been established, open the Workspace Configuration section and select **Save As...**.

Enter a descriptive workspace name, such as:

```
RF Measurements
```

Confirm the save operation.

The controller stores the workspace configuration on the microSD card, and the new workspace becomes available in the workspace selector.

A workspace name should describe its intended purpose rather than simply identifying the instruments it contains. Names such as RF Measurements, Spectrum Analysis, and Receiver Testing can help distinguish different operating arrangements.

6.4 Saving Changes to an Existing Workspace

After loading a saved workspace, the operator may change its configuration.

For example, the operator might select a different function on the spectrum analyzer, modify displayed parameter values, or add another Control Panel.

To preserve these changes in the currently loaded workspace, select **Save Configuration**. The existing saved configuration is updated to reflect the current browser arrangement.

If the operator wishes to retain the original workspace and create a second version, use **Save As...** instead.

For example, an existing workspace named RF Measurements could be loaded, modified, and saved under the new name:

```
RF Measurements - Sweep
```

This permits several related workspace configurations to be maintained without repeatedly rebuilding the Control Panels.

It is important to distinguish between a workspace that has merely been selected in the workspace selector and one that has actually been loaded. Selecting a workspace name does not, by itself, make that workspace the active configuration. Use **Load** before modifying and saving an existing workspace.

6.4.1 Unsaved Changes and Reverting a Configuration

Whenever the operator modifies the **current** workspace, the controller displays an orange **Unsaved Changes** indicator in the Workspace Configuration section (see Figure 6.1 above). This provides a visual reminder that the current browser arrangement differs from the last saved configuration.

Changes that may cause this indicator to appear include adding or removing Control Panels, selecting different instruments or functions, and modifying displayed parameter values.

The operator has several choices:

- **Save Configuration** saves the current arrangement to the existing workspace.
- **Save As...** preserves the current arrangement under a new workspace name.
- **Revert Configuration** discards unsaved changes and restores the last saved configuration.

The **New Workspace** button provides a way to begin constructing a new workspace rather than modifying an existing one.

The Unsaved Changes indicator disappears when the current configuration has been saved or reverted to its previously saved state.

It is important to distinguish between changes to the browser workspace and changes to connected instruments. **Reverting a workspace does not undo commands already sent to instruments.** Similarly, saving or loading a workspace does not send commands to the connected instruments.

6.5 Loading a Saved Workspace

To restore a previously saved workspace, open the Workspace Configuration section and select the desired configuration from the workspace selector. Select **Load**.

The controller restores the saved browser arrangement, including the Control Panels, selected instruments, functions, and displayed parameter values.

If Macro Panels were included in the saved configuration, their arrangements are also restored.

For example, loading RF Measurements might restore the Marconi signal generator and E4407B spectrum analyzer in their previously selected functions.

The operator can then continue working with the restored panels.

Note: Loading a workspace does not automatically send the saved parameter values to the connected instruments.

This distinction is important because an instrument's actual settings may have changed since the workspace was saved.

For example, the saved Marconi Control Panel might display a frequency of 146 MHz, while the generator itself is currently operating at 150 MHz. Loading the workspace restores the displayed value of 146 MHz, but it does not change the generator's output frequency.

To determine the actual frequency, select the corresponding **Read** button. To command the generator to the displayed value, select **Set**.

This behavior prevents unexpected instrument changes when switching between workspaces.

6.6 Renaming and Deleting Workspaces

The Workspace Configuration section also provides controls for renaming and deleting saved configurations.

6.6.1 Renaming a Workspace

To rename a workspace, select the desired configuration from the workspace selector and choose **Rename**. Enter the new name and confirm the operation.

Renaming is useful when a workspace's purpose has changed or when a more descriptive name is desired.

For example: `RF Measurements` might be renamed: `Marconi-E4407B Measurements.`

Renaming changes the saved workspace's identification without requiring the operator to recreate its Control Panels.

6.6.2 Deleting a Workspace

To remove a workspace that is no longer required, select it from the workspace selector and choose **Delete**. Confirm the deletion when requested.

Deleting a workspace removes its saved configuration from the microSD card. It does not delete the instrument inventory, instrument profiles, or macro definitions referenced by that workspace.

For example, deleting a workspace containing the Marconi 2024 does not remove the Marconi instrument definition from `/config/instruments.cfg` or its profile from `/instruments`.

The instrument remains available for selection in other workspaces.

Before deleting a workspace, consider whether it contains a useful configuration that would be difficult to recreate.

6.7 Selecting a Startup Workspace

The controller can automatically restore a selected workspace when its browser interface is opened. This is particularly useful when the controller is normally used with the same instruments and Control Panel arrangement.

For example, an operator who routinely uses a Marconi 2024 and E4407B might wish to have both Control Panels available immediately whenever the controller interface is opened.

To designate a startup workspace, select the desired saved configuration in the Workspace Configuration section.

Enable the checkbox labeled:

Load this configuration when the controller opens

The controller records the startup workspace selection on the microSD card.

The next time the controller interface is opened, the designated workspace is restored automatically.

The startup workspace is identified by the file: `/config/startup_workspace.txt`

This file records the name of the workspace selected for automatic loading.

The startup workspace selection is separate from the workspace's contents. A workspace may be edited and saved without necessarily changing which workspace is designated for startup.

As with manually loading a workspace, automatic restoration does not transmit the saved parameter values to connected instruments.

The workspace designated for automatic startup is identified by an asterisk (*) appended to its filename in the Workspace Configuration selection menu. For example, if RF Measurements has been selected as the startup workspace, it will appear as **RF Measurements*** in the list of saved configurations. This provides a convenient visual indication of which workspace will be loaded automatically when the controller interface is opened.

Note that the asterisk is only a visual indicator; it is not part of the actual workspace filename.

6.8 Workspace Files on the microSD Card

Saved workspace configurations are stored in the directory:

```
/config/workspaces
```

This directory is created automatically when the first workspace is saved.

The startup workspace selection is stored separately in:

```
/config/startup_workspace.txt
```

Because these files reside on the microSD card rather than in the browser, workspace configurations are retained independently of the computer used to access the controller. For example, a workspace saved from a desktop computer can subsequently be loaded from a laptop or another device accessing the same controller.

The files also remain available after the controller is powered off and restarted.

Workspace files should be included when backing up the controller's microSD card, particularly when they contain arrangements that required considerable effort to configure.

However, workspace files do not replace the instrument inventory, instrument profiles, or macro definitions. A workspace that references an instrument or macro depends on the corresponding configuration files remaining available.

For a complete backup, preserve the relevant contents of `/config`, `/instruments`, and `/macros`.

6.9 Workspaces and Macro Panels

Workspace configurations can include both instrument Control Panels and Macro Panels. This permits a measurement arrangement to be saved as a complete working environment.

For example, a workspace for automated frequency-response measurements might contain:

- A Marconi 2024 Control Panel for setting the RF signal generator.
- An E4407B Control Panel for observing the spectrum analyzer.
- A Macro Panel containing the FrequencySweep macro.

The operator can use the Control Panels to establish or verify instrument settings and then execute the macro to perform the automated measurement.

When the workspace is saved, its panel arrangement and selections are preserved.

The macro definition itself remains in the `/macros` directory. Saving a workspace does not create a separate copy of the macro.

Consequently, if a macro file is deleted or renamed, a previously saved workspace referencing that macro may no longer be able to restore the expected selection.

The same principle applies to instrument definitions and profiles.

6.10 Practical Considerations

Workspace management is intended to simplify operation, but several points should be kept in mind.

First, a workspace is not an instrument preset. Although it can preserve displayed values, it does not guarantee that the connected instruments have those values when the workspace is restored.

Second, workspace configurations depend on the availability of the instruments and macros they reference. If the instrument inventory is changed, or if profile files are removed, some saved selections may no longer be valid.

Third, workspaces can be used to organize different measurement procedures involving the same instruments. There is no need to create separate instrument definitions merely to obtain different Control Panel arrangements.

Finally, workspace files should be backed up along with the other controller configuration files. This is especially useful before making extensive changes to instrument profiles, macro definitions, or the microSD card contents.

6.11 Summary

The Workspace Configuration facility provides a convenient means of preserving and restoring the browser arrangements used for different measurement procedures. The operator can create, save, load, rename, and delete workspaces, and designate one configuration to load automatically when the controller interface opens.

A workspace can include multiple instrument Control Panels and Macro Panels, making it possible to organize complex measurement setups without repeatedly selecting instruments and functions.

Because workspace configurations are stored on the microSD card, they are retained across controller restarts and can be accessed from different computers.

The distinction between displayed parameter values and actual instrument settings remains important. Restoring a workspace recreates the browser configuration but does not automatically command the connected instruments to the saved values.

7. File Management and System Maintenance

7.1 Introduction

The AK3Y Instrument Controller uses a microSD card to store instrument definitions, instrument profiles, macros, workspace configurations, and diagnostic information. These files are separate from the controller firmware and can generally be created or modified without recompiling the application.

The File Management screen provides browser-based access to the microSD card. Through this interface, the operator can create directories, upload files, examine their contents, edit supported text files, download files to a computer, and delete files that are no longer needed.

This capability is particularly useful when the controller is installed in an equipment rack or another location where physically removing the microSD card would be inconvenient.

Figure 2-3 provided an overview of the File Management screen. This chapter explains the individual operations in greater detail.

7.2 Organization of the microSD Card

The controller organizes its files into four principal directories, which are created automatically during startup if they do not already exist.

Table 7-1. Principal microSD Card Directories

Directory	Purpose
`/config`	Instrument inventory and controller configuration files
`/instruments`	Individual instrument profile files
`/macros`	Macro definitions
`/logs`	Diagnostic information and measurement data files

The `/config` directory contains the instrument inventory file, `instruments.cfg`, which identifies the available instruments and their communications settings. It also contains workspace-related information. Saved workspaces are stored in `/config/workspaces`, while `/config/startup_workspace.txt` identifies the workspace designated for automatic loading.

The `/instruments` directory contains the individual instrument profiles described in Appendix B. For example, `marconi_2024.cfg` defines the controls and commands used to operate the Marconi 2024 signal generator.

The `/macros` directory contains the macro definitions described in Chapter 5 and Appendix C. A file named `FrequencySweep.cfg`, for example, appears in the Macro selection menu as FrequencySweep.

Finally, the `/logs` directory contains diagnostic information and files produced by automated measurements.

The four principal system directories are protected against deletion through the browser interface. Additional directories may be created by the operator when needed.

7.3 Creating Directories

The upper section of the File Management screen is labeled **Create SD Card Directory**. This facility allows the operator to create a new directory within an existing directory. For example, an operator might wish to create a directory for retaining copies of older instrument profiles.

To create a directory, select the desired parent directory from the Parent Folder menu. Enter the name of the new directory in the New directory name field and select Create Directory. The controller creates the directory on the microSD card. The new directory should then appear in the file listing.

Directory names should be descriptive and should avoid characters that may cause difficulties when files are subsequently accessed or transferred to another computer.

The four principal system directories should normally be left unchanged. The controller expects to find instrument profiles, macros, and configuration files in their designated locations.

7.4 Uploading Files

The **SD Card File Upload** section allows files to be transferred from the operator's computer to the controller's microSD card. This is the normal method for installing new instrument profiles, macro definitions, and configuration files.

7.4.1 Selecting a File

First, select the destination directory from the SD Card File Upload section.

Next, use the file-selection control to choose a file from the computer. The interface also provides a field for specifying the name under which the file will be stored on the microSD card.

For example, to install the Marconi 2024 instrument profile, select `/instruments` as the destination directory and choose `marconi_2024.cfg`.

The stored filename should normally be left unchanged, particularly when the file is referenced by another controller configuration file.

Select **Upload to SD Card** to begin the transfer. After the upload has completed, confirm that the file appears in the appropriate directory.

7.4.2 Upload Size Limit

The File Management interface supports uploads of up to **256 KB per file**.

This limit applies to the individual file being transferred, not to the total capacity of the microSD card.

Instrument profiles and macro definitions are normally much smaller than this limit. However, large text files or other files may exceed it.

If an upload fails because the file exceeds the permitted size, the controller displays an error rather than accepting the file. A file larger than 256 KB should be transferred by another suitable method, such as copying it directly to the microSD card using a computer.

7.4.3 Replacing an Existing File

If a file with the same name already exists in the selected destination directory, the controller requests confirmation before replacing it.

Before confirming the replacement, make sure that the correct file and destination directory have been selected. It is good practice to download a copy of an existing configuration file before replacing it, particularly when the file contains modifications that may be difficult to reconstruct.

7.4.4 When Do Uploaded Changes Take Effect?

Uploading a file changes the contents of the microSD card, but it does not necessarily change the configuration already loaded into the controller's memory.

For example, after replacing `/config/instruments.cfg`, the controller must be restarted so that the revised instrument inventory is loaded.

The same conservative procedure is recommended after changing instrument profiles.

Macro files are separate from the firmware and can be managed through the File Management screen. After replacing a macro, reopen or reselect it before attempting to execute the revised definition.

When there is uncertainty about whether a modified configuration has been reloaded, restarting the controller provides a straightforward way to ensure that its startup configuration is read again.

7.5 Viewing and Editing Files

The lower section of the File Management screen, labeled **SD Card Files**, displays the files and directories stored on the microSD card.

Files are organized by directory. Each file entry includes its filename and size, together with the operations available for that file.

Depending on the file type, these operations may include **View**, **Edit**, **Download**, and **Delete**.

7.5.1 Viewing a File

Select View beside a file to examine its contents in the browser. This is particularly useful for checking instrument profiles, macro definitions, and diagnostic logs without first downloading them to a computer.

For example, the operator can examine `/config/instruments.cfg` to verify that the Marconi signal generator is configured for GPIB address 20.

Viewing a file does not modify its contents. For very large files, downloading the file and opening it with a text editor may be more convenient than viewing it through the browser.

7.5.2 Editing a File

For supported text files, the Edit control allows the contents to be modified directly through the browser. This is convenient for making relatively small changes to instrument profiles or macros.

For example, suppose the operator wishes to change the maximum permissible frequency in an instrument profile. Rather than editing the file on a computer and uploading a replacement, the operator can open the file using Edit, modify the appropriate parameter, and save the revised contents. The controller stores the changes on the microSD card.

Because configuration files are interpreted according to specific syntax rules, care should be taken when making changes. Incorrect section names, parameter names, or command strings can prevent the controller from interpreting the file correctly.

For substantial changes, editing a downloaded copy on a computer is often preferable. This provides an opportunity to retain the original file and compare revisions before uploading the replacement.

Editing a file on the microSD card does not necessarily update a configuration already loaded into memory. Restart the controller when changing its instrument inventory or profiles.

7.6 Downloading and Deleting Files

7.6.1 Downloading a File

The Download control transfers a copy of the selected file from the controller's microSD card to the operator's computer.

This is useful for creating backups, examining diagnostic information, and retrieving measurement results.

For example, the FrequencySweep macro described in Chapter 5 creates a measurement file such as `/logs/frequency_sweep.csv`

After the macro has completed, the operator can open the File Management screen, locate the file in `/logs`, and select Download. The downloaded CSV file can then be opened using spreadsheet or data-analysis software.

Downloading a file does not remove it from the controller.

7.6.2 Deleting a File

The Delete control removes the selected file from the microSD card. This operation is useful for removing obsolete macro definitions, superseded instrument profiles, or measurement files that are no longer required.

However, deleting a configuration file can affect other controller functions.

For example, deleting `marconi_2024.cfg` from `/instruments` would leave any inventory entry referencing that profile without its required definition.

Similarly, deleting a macro file would make that macro unavailable for subsequent selection and execution.

Before deleting a file, verify that it is no longer needed and consider downloading a backup copy.

7.6.3 Deleting Directories

The File Management interface also permits the removal of empty directories that are not protected system directories.

A directory containing files must first be emptied before it can be removed.

The four principal system directories, namely`/config`, `/instruments`, `/macros`, and `/logs`—cannot be deleted through the browser interface.

7.7 Diagnostic and Measurement Log Files

The `/logs` directory serves two distinct purposes: retaining diagnostic information about controller operation and storing measurement results produced by macros.

7.7.1 Controller Diagnostics

The controller maintains a diagnostic log in the `/logs` directory. The active diagnostic file is named `/logs/diagnostics.txt`. This file records information useful for investigating controller startup, initialization, and operating problems.

To prevent the active diagnostic file from growing indefinitely, the controller automatically rotates diagnostic logs when `diagnostics.txt` reaches 256 KB (262,144 bytes). The existing log is archived, and a new `diagnostics.txt` file is created. Previously archived logs are

retained, with numerical suffixes (`diagnostics-1.txt`, `diagnostics-2.txt`, etc.) identifying successive generations.

There is no programmed limit on the number of archived diagnostic files. Consequently, the total space occupied by diagnostic logs can continue increasing until the available microSD card storage is exhausted. Periodic removal of obsolete archives is therefore recommended.

The operator can examine these files using View or download them for more detailed inspection.

Archived logs are particularly useful when investigating problems that occurred before the most recent rotation, including earlier controller restarts or initialization failures.

When reporting a problem, it is advisable to preserve the current diagnostic log and any relevant archived logs, together with the firmware version shown on the About screen.

Diagnostic files that are no longer needed can be deleted through File Management.

7.7.2 Measurement Files

Macros can create measurement files using the LOG command described in Appendix C.

For example, the FrequencySweep macro contains an instruction of the form:

```
LOG /logs/frequency_sweep.csv Freq_IN,Freq_Measured,Power_Measured
```

This command records the commanded frequency, measured frequency, and measured amplitude in a CSV file.

When the file is initially empty, the macro writes a header identifying the recorded variables. Subsequent measurement records are appended to the file.

The resulting file can be downloaded and examined using a spreadsheet or other analysis program.

An important consequence of appending records is that repeated execution of a macro may add measurements to an existing file rather than replacing its previous contents. If a new measurement run must begin with an empty file, the operator should first download the existing file if it is to be retained, then delete it before running the macro again.

7.7.3 Managing Log Files

Because diagnostic log rotation does not impose a limit on the number of archived files, the `/logs` directory should be reviewed periodically.

Although each individual diagnostic file is limited by the rotation threshold, the combined size of the archived files can continue increasing as the controller operates. Measurement files created by macros may also accumulate, particularly when repeated measurements are appended to existing CSV files.

The operator should periodically examine the `/logs` directory, download files that should be retained, and delete obsolete files to recover storage space.

For example, diagnostic archives from an earlier troubleshooting session may no longer be needed once the problem has been resolved. Similarly, measurement files that have already been transferred to a computer may be removed from the controller.

Care should be taken not to delete diagnostic information that may still be useful for investigating unresolved problems.

Automatic log rotation does not replace periodic file maintenance. Since the number of archives is not limited, the operator remains responsible for ensuring that sufficient free space is available on the microSD card.

7.8 Backing Up and Restoring the microSD Card

The controller's configuration is largely determined by the files stored on its microSD card. Consequently, maintaining a backup of these files is an important part of system maintenance.

A useful backup should include the instrument inventory, individual instrument profiles, macros, and saved workspaces.

Table 7-2. Recommended Backup Contents

Files or directory	Reason for backup
`/config/instruments.cfg`	Preserves instrument definitions and interface settings
`/config/workspaces`	Preserves saved Control Panel and Macro Panel arrangements
`/config/startup_workspace.txt`	Preserves the startup workspace selection
`/instruments`	Preserves instrument-specific commands and controls
`/macros`	Preserves automated measurement procedures
`/logs`	Preserves diagnostic information and measurement results when required

Individual files can be downloaded through the browser interface. However, for a complete backup, it may be more convenient to remove the microSD card after powering down the controller and copy its contents using a computer.

A replacement microSD card should be formatted using FAT32, as described in Section 3.5.

The saved directories and files can then be copied to the replacement card.

After installing the card and restarting the controller, verify that the instrument inventory, profiles, macros, and workspaces are available.

Note that Wi-Fi credentials are stored in the ESP32's nonvolatile memory rather than on the microSD card. Consequently, copying the microSD card does not transfer the saved Wi-Fi credentials to another controller.

7.9 File Management Errors and Recovery

Most File Management problems can be resolved by checking the selected directory, filename, file size, and microSD card status.

Table 7-3. Common File Management Problems

Problem	Suggested action
File upload fails	Confirm that the file does not exceed 256 KB and that the microSD card is available
File appears in the wrong directory	Upload it again to the correct destination and remove the misplaced copy if appropriate
Uploaded instrument does not appear	Verify `/config/instruments.cfg`, the referenced profile, and restart the controller
Macro does not appear in the selection menu	Confirm that its `.cfg` file is in `/macros`
File cannot be viewed or edited	Download it and inspect it using a suitable application
Measurement file contains earlier results	Check whether the macro has appended records from previous executions
Workspace cannot be restored correctly	Verify the saved workspace and the instrument and macro files it references
Diagnostic archives occupy excessive space	Download or delete older `diagnostics-N.txt` files
microSD card is nearly full	Remove obsolete measurement files and diagnostic archives after preserving any needed records
microSD card is not recognized	Check the card installation and FAT32 formatting

When a file operation fails, the operator should read any error message displayed by the browser before attempting the operation again.

If the controller's microSD card is not detected, the About screen can be used to confirm the reported card status.

A persistent card problem may require powering down the controller, checking the card connection, and testing the card using a computer.

7.10 Summary

The File Management screen provides a convenient means of maintaining the controller's microSD card without physically removing it.

The operator can create directories, upload new files, view and edit supported text files, download measurement results, and delete obsolete files.

The four principal system directories organize instrument definitions, profiles, macros, and logs. Their contents determine much of the controller's behavior and should be backed up regularly.

The controller automatically rotates diagnostic logs when the active file reaches its size threshold. Archived files are retained with sequential numerical suffixes, and there is no programmed limit on their number. Periodic removal of obsolete archives and measurement files is therefore important to maintain sufficient storage space.

Particular care should be taken when replacing instrument configuration files, because changes may require restarting the controller before they take effect.

Together with the workspace facilities described in Chapter 6, File Management allows the AK3Y Instrument Controller to be adapted to new instruments and measurement procedures without modifying the firmware.

8. Network Configuration, Firmware Updates, and Troubleshooting

8.1 Introduction

The AK3Y Instrument Controller incorporates a Wi-Fi interface that allows the operator to control instruments, execute macros, and manage files through a standard web browser. Once the controller has been connected to a local wireless network, no special software is required on the computer used to operate it.

The controller also supports firmware updates through its web interface. This permits new firmware versions to be installed without connecting the controller directly to a development computer.

This chapter describes normal network operation, procedures for changing Wi-Fi networks, firmware maintenance, and methods for diagnosing common operating problems.

The initial installation of the firmware and first-time Wi-Fi configuration are described in Chapter 3. The following sections expand on those procedures and address situations that may arise during normal operation.

8.2 Normal Network Operation

When power is applied, the controller initializes its hardware, mounts the microSD card, and attempts to connect to the wireless network whose credentials are stored in the ESP32's nonvolatile memory.

If the connection succeeds, the wireless network assigns an IP address to the controller. This address is displayed on the controller's OLED screen.

To access the controller, open a web browser on a computer, tablet, or other device connected to the same local network and enter the displayed IP address.

For example, if the OLED displays: `192.168.1.150`, enter the following address in the browser: `http://192.168.1.150/`.

The controller's web interface should appear, providing access to the Instruments, Macros, File Management, and About screens.

The assigned IP address may change when the controller reconnects to the network, particularly if the router uses DHCP to allocate addresses dynamically. Consequently, the OLED display should be consulted whenever the controller cannot be reached using a previously recorded address.

8.2.1 Network Requirements

The controller and the computer used to operate it must be able to communicate over the local network. Normally, this means that both devices are connected to the same home or laboratory network.

Some wireless routers provide a separate guest network that prevents connected devices from communicating with one another. Such a network may not be suitable for operating the controller.

Similarly, firewalls or network isolation settings may prevent access to the controller even when the Wi-Fi connection itself has been established.

The controller communicates with instruments equipped with TCP interfaces through the local network. Those instruments must also be reachable at the host addresses and port numbers specified in the instrument inventory file.

8.2.2 Network Status Information

The controller provides several indications of its network status. The OLED displays the assigned IP address and other operating information. The browser's status header reports the Wi-Fi connection and signal strength. Additional information is available on the About screen, including:

- Wi-Fi connection status
- Network name (SSID)
- Controller hostname
- Assigned IP address
- Controller uptime
- Firmware version

The About screen reports the controller's status at the time the page was loaded. Refreshing the page obtains updated information.

These indications are useful when investigating intermittent network connections or determining whether the controller has connected to the expected wireless network.

8.3 Changing the Wi-Fi Network

The controller provides a browser-based procedure for changing its wireless network without modifying or reinstalling the firmware.

This procedure is useful when the controller is moved to another location, a wireless router is replaced, or the network name or password changes.

The Update Wi-Fi screen is illustrated in Figure 2-6.

8.3.1 Entering Wi-Fi Setup Mode

Before beginning, make sure that the computer or mobile device being used can connect to the new wireless network.

From the controller's web interface, select **Update Wi-Fi**.

The page displays instructions for the configuration procedure. Read these instructions before proceeding, because the existing browser connection will be interrupted.

Select **Enter Setup Mode**. The controller temporarily disconnects from its normal wireless network and establishes its own wireless access point.

The access point is named:

```
AK3Y-Controller-Setup
```

No password is required to join this temporary network.

8.3.2 Connecting to the Setup Access Point

On the computer, tablet, or smartphone being used for configuration, open the list of available wireless networks.

Select `AK3Y-Controller-Setup`. After connecting, open a web browser and enter:

```
http://192.168.4.1/
```

This address accesses the controller's temporary network configuration page. The page allows the operator to select a wireless network and enter its password.

If the desired network is not shown in the list of detected networks, its name may be entered manually.

8.3.3 Saving the New Network Configuration

Select the desired wireless network and enter the appropriate password. Select **Test and save network**. The controller attempts to establish a connection using the supplied credentials. The credentials are saved only after a successful connection test.

When the new network has been accepted, reconnect the computer or mobile device to that network. The controller's new IP address is displayed on its OLED screen. Enter this address in the browser to resume normal operation.

Because the controller may receive a different IP address on the new network, a previously saved browser address may no longer be valid.

8.3.4 Setup Timeout and Cancellation

When Wi-Fi configuration is initiated from the normal web interface, the setup session is limited to five minutes.

If the operator cancels the procedure or the session times out, the controller restores its previously saved network configuration.

This provides a means of returning to the original network if the operator does not complete the configuration procedure.

The five-minute timeout applies only to a network change initiated from the normal web interface. During initial installation, when the controller has no usable saved Wi-Fi credentials, the initial setup procedure does not have this time limit.

8.3.5 Storage of Wi-Fi Credentials

Wireless network credentials are stored in the ESP32's nonvolatile memory rather than on the microSD card. Consequently, replacing or reformatting the microSD card does not ordinarily remove the saved Wi-Fi configuration.

Similarly, copying the contents of a microSD card to another controller does not transfer the wireless credentials.

Each controller must establish its own network configuration.

8.4 Firmware Updates

The controller supports over-the-air (OTA) firmware updates through its web interface.

This facility allows a new firmware version to be installed without physically connecting the controller to a computer through USB.

The Firmware Update screen is illustrated in Figure 2-5.

8.4.1 Obtaining a Firmware Image

Firmware updates are distributed as compiled ESP32 application images with the extension:

```
.bin
```

The appropriate firmware image should be obtained from the AK3Y Instrument Controller project repository or another authorized distribution source.

The image must have been compiled for the controller's ESP32 hardware and firmware partition arrangement. A firmware image intended for another ESP32 board or an incompatible partition layout should not be installed.

Note: The firmware image used for an OTA update is not necessarily the same as a complete initial-programming package.

As explained in Section 3.4, initial programming may require additional flash components, including the bootloader and partition table.

For browser-based updating, select the application firmware image specifically intended for OTA installation.

8.4.2 Checking the Installed Firmware

Before installing an update, open the About screen. The page identifies the currently installed firmware version and build date. It also displays system information, including firmware image size and available application storage space.

Record the installed firmware version before proceeding. This information is useful if it becomes necessary to compare the behavior of the updated controller with that of the previous release.

The Firmware Update screen also displays information about the current firmware and available OTA application space.

8.4.3 Installing an Update

To install a firmware update:

Open the controller's web interface and select **Firmware Update**.

Review the currently installed firmware information and available OTA space.

Select **Select Firmware File...**

Choose the appropriate .bin file from the computer.

Confirm that the selected file is the intended firmware image.

Select Install Firmware to begin the update.

The controller receives the firmware image and installs it in its application flash memory.

Important! Do not interrupt power to the controller while a firmware update is in progress.

Do not deliberately disconnect the controller from the network or close the browser during the transfer and installation process.

When installation is complete, the controller restarts to execute the newly installed firmware. The browser connection may be temporarily interrupted during this restart.

8.4.4 Verifying the Update

Allow the controller sufficient time to restart and reconnect to its wireless network.

Observe the OLED display and note the IP address.

Reconnect to the controller using the browser. If the previous browser session does not reconnect automatically, reload the page using the displayed IP address.

Open the About screen and confirm that the firmware version corresponds to the newly installed release.

Also verify that:

- The microSD card is detected.
- The controller has connected to the expected Wi-Fi network.
- The Instruments screen displays the configured instruments.
- Saved workspaces and macros remain available.
- Basic instrument communications operate correctly.

The firmware is stored in the ESP32's flash memory, while instrument profiles, macros, and workspace configurations are stored separately on the microSD card.

A normal firmware update does not require replacing these files unless the release instructions specifically identify a configuration change. Nevertheless, it is good practice to maintain a backup of the microSD card before installing a new firmware version.

8.4.5 Firmware Update Problems

If the firmware file is rejected, verify that the selected file is a valid ESP32 application image and that it is compatible with the controller.

Also check the available OTA application space reported on the Firmware Update screen.

If the update fails before installation is complete, read the error information displayed by the browser.

If the browser loses contact with the controller during a restart, check the OLED display before assuming that the update has failed. The controller may simply be reconnecting to Wi-Fi.

If the controller no longer starts correctly or cannot provide its normal web interface, recovery may require programming the ESP32 through its USB connection, using the procedures described in Chapter 3.

8.5 Using the About Screen for Diagnostics

The About screen provides a convenient summary of the controller's current operating condition. Figure 2-4 illustrates the information presented.

The upper portion identifies the controller, firmware version, and build information.

The System Information section includes several values useful for troubleshooting.

Table 8-1. About Screen Diagnostic Information

Item	Significance
Firmware version	Identifies the installed software release
Firmware build date and time	Identifies when the firmware image was compiled
Uptime	Time elapsed since the controller last restarted
Free heap	Available dynamic memory
Firmware image size	Size of the installed application
Available application space	Storage available for application updates
Flash capacity	Total installed flash-memory capacity
Wi-Fi status	Indicates whether the controller is connected to a wireless network
SSID	Identifies the connected wireless network
Hostname	Network name assigned to the controller
IP address	Address used to access the browser interface
SD card	Indicates whether the microSD card has been detected

The reported values are snapshots. Refreshing the About screen obtains current information.

8.5.1 Controller Uptime

Uptime is particularly useful when investigating unexpected restarts. For example, if the controller has been operating continuously for several days, its reported uptime should reflect that interval.

An unexpectedly short uptime may indicate that the controller has restarted because of a power interruption, manual reset, firmware update, or another event.

The diagnostic log files described in Section 7.7 may help identify the circumstances surrounding the restart.

8.5.2 Available Memory

The free heap value indicates the amount of dynamic memory currently available to the application. The reported value may vary during normal operation as the controller processes browser requests, executes macros, or performs other activities.

An unusually low value, particularly when accompanied by operating problems, may be useful information when investigating firmware behavior.

For ordinary operation, the user generally does not need to monitor this value continuously.

8.5.3 microSD Card Status

The SD card indication confirms whether the controller has detected and mounted its microSD card.

If the card is not detected, instrument profiles, macros, saved workspaces, and diagnostic files may be unavailable.

Check the card installation and FAT32 formatting, as described in Sections 3.5 and 7.9.

8.6 Troubleshooting Network Problems

Most network problems involve incorrect credentials, an unexpected IP address, weak wireless reception, or restrictions imposed by the local network.

The following procedure provides a systematic approach to identifying the cause.

8.6.1 Controller Cannot Be Reached from the Browser

First, examine the controller's OLED display.

If an IP address is displayed, confirm that the computer is connected to the expected local network.

Enter the IP address directly into the browser, including the `http://` prefix if necessary.

If the controller still cannot be reached, verify that the computer is not connected to an isolated guest network.

Check whether the router has assigned a different IP address since the previous session.

If necessary, restart the controller and observe its network status during startup.

8.6.2 Controller Does Not Connect to Wi-Fi

If the controller cannot establish its normal wireless connection, verify that the configured network is available and that the password has not changed.

Check that the controller is within suitable range of the wireless access point.

If the network credentials are no longer valid, use the Wi-Fi setup procedure described in Section 8.3.

When the controller has no usable saved network connection, it enters setup mode so that new credentials can be supplied.

8.6.3 Browser Connection Is Intermittent

An intermittent browser connection may be caused by weak Wi-Fi reception or network congestion. Check the Wi-Fi signal indication in the controller's status header.

If the controller is located inside a metal equipment rack or near substantial metal structures, wireless reception may be reduced. Moving the controller or improving the wireless access point's coverage may help.

If the connection remains unreliable, determine whether other devices on the same network are experiencing similar problems.

8.6.4 TCP Instruments Cannot Be Reached

A successful connection between the browser and the AK3Y Instrument Controller does not necessarily mean that the controller can communicate with a separate TCP instrument.

Verify the instrument's host address and port number in `/config/instruments.cfg`.

Confirm that the instrument is powered on, connected to the network, and configured to accept remote commands. Also check whether the instrument's network address has changed.

For TCP instruments, the controller must be able to establish a connection to the specified host and port independently of the browser connection.

8.7 Troubleshooting Instrument Communications

Instrument communications problems should be distinguished from browser or Wi-Fi problems.

The browser may be functioning normally even when an attached GPIB, RS-232, or TCP instrument is not responding.

Likewise, a Control Panel may appear correctly because its controls are generated from a stored profile, even though the physical instrument is disconnected.

8.7.1 GPIB Instruments

For a GPIB instrument that does not respond:

Confirm that the instrument is powered on.

Check the GPIB cable and connector.

Verify that the instrument is configured for GPIB operation.

Confirm that its GPIB address matches the address specified in `/config/instruments.cfg`.

Test a simple command or query known to be supported by the instrument.

Some instruments provide more than one remote-control interface. Selecting RS-232 operation on the instrument may disable its GPIB interface. The controller may then report a communications error even though the GPIB cable and address are correct.

8.7.2 RS-232 Instruments

For an RS-232 instrument, verify the physical cable connections and the communications parameters specified in the instrument inventory. These include the controller port, baud rate, data bits, parity, and stop bits. The settings must agree with those configured on the instrument. A mismatch may prevent communication entirely or cause responses to contain unexpected characters.

Particular attention should be paid to the RS-232 cable configuration. **Most instruments will require a null-modem cable or adapter** to communicate with the AK3Y Instrument Controller. A null modem crosses the transmit (TXD) and receive (RXD) connections, allowing each device's transmitter to connect to the other device's receiver. The signal-ground connection remains straight through.

The need for a null modem depends on whether the instrument is configured as Data Terminal Equipment (DTE) or Data Communications Equipment (DCE). If both devices use the same configuration, a null modem is required; if they use opposite configurations, a straight-through cable is appropriate. Consult the instrument's documentation to determine the correct wiring.

8.7.3 TCP Instruments

For TCP instruments, confirm the host address and port number.

Verify that the instrument is accessible from the controller's network and that the instrument supports the command termination and response conventions specified in its profile.

The instrument may accept a network connection but still fail to interpret commands if the required terminators are incorrect.

8.7.4 Instrument Profile Errors

When communications appear to be established but a particular command fails, examine the corresponding instrument profile.

For example, a Set operation may succeed while a Read operation fails because the profile specifies an incorrect query command, response prefix, or response terminator.

The profile syntax and supported parameters are described in Appendix B.

The Advanced Command Console, described in Appendix D, can also be useful for testing individual operations.

8.8 Troubleshooting Macros and Workspaces

8.8.1 Macro Does Not Appear

If a macro is missing from the Macro selection menu, verify that its `.cfg` file is stored in `/macros`.

The displayed macro name is derived from the filename, without the `.cfg` extension.

For example, `FindSignal.cfg` appears in the selection menu as:

```
FindSignal
```

Check the file's syntax if it is present but cannot be interpreted correctly.

8.8.2 Macro Execution Fails

A macro may fail because an instrument is unavailable, a capability name is incorrect, a command is rejected, or a required response is not received.

Begin by confirming that the instruments referenced by the macro are defined in the instrument inventory.

Next, verify that the referenced capabilities exist in the corresponding instrument profiles.

If the macro fails during a measurement, try performing the same operation manually from an instrument Control Panel.

This can help distinguish a communications problem from an error in the macro instructions.

The macro command language is described in Appendix C.

8.8.3 Saved Workspace Does Not Match Instrument Settings

Loading a workspace restores the browser's Control Panels, Macro Panels, selected functions, and displayed parameter values.

It does not transmit the saved values to the connected instruments.

Consequently, the values shown after loading a workspace may differ from the instruments' actual settings.

Use the Read controls to obtain current instrument values, or deliberately select Set to apply the displayed values.

This behavior is normal and is explained in Chapter 6.

8.8.4 Unsaved Workspace Changes

The Workspace Configuration section displays **Unsaved Changes** when the current browser configuration differs from the last saved configuration.

Use Save Configuration to preserve the changes, Save As... to create a separate workspace, or Revert Configuration to restore the previously saved arrangement.

Reverting a workspace does not undo commands already transmitted to physical instruments.

8.9 Diagnostic Logs and Problem Reporting

The controller maintains diagnostic information in the `/logs` directory of its microSD card.

The active diagnostic log is:

```
/logs/diagnostics.txt
```

As described in Section 7.7, the controller automatically rotates this file when it reaches 256 KB.

Archived files are named `diagnostics-1.txt`, `diagnostics-2.txt`, and so forth.

There is no programmed limit on the number of archived files; the practical limit is the available microSD card space.

When investigating a persistent problem, examine the active log and any relevant archived files.

The File Management screen permits these files to be viewed or downloaded.

When reporting a problem, the following information is particularly useful:

- Firmware version and build date
- Description of the problem and the operation being attempted
- Whether the problem is reproducible
- Instrument manufacturer, model, and communications interface
- Relevant instrument inventory and profile files

- Macro file, if the problem occurs during macro execution
- Current and relevant archived diagnostic logs

A useful troubleshooting procedure is to begin with the simplest operation that reproduces the problem.

For example, if a macro involving two instruments fails, first verify that each instrument can be operated independently through its Control Panel.

Then test the individual capabilities used by the macro before attempting the complete automated sequence.

This approach generally makes it easier to identify whether the difficulty involves network access, instrument communications, command interpretation, or macro execution.

8.10 Troubleshooting Summary

Table 8-2. Common Operating Problems

Problem	Suggested action
Browser cannot reach controller connection	Check OLED IP address and local network
Controller cannot connect to Wi-Fi	Verify network availability and credentials; enter setup mode if necessary
Wi-Fi connection is intermittent	Check signal strength and wireless coverage
Controller receives a different IP address	Use the current address displayed on the OLED
Firmware update file is rejected	Verify that the `.bin` image is compatible and fits the available OTA space
Browser disconnects during firmware restart	Wait for the controller to reconnect and check the OLED
microSD card is not detected	Check card installation and FAT32 formatting
Instrument does not appear	Check `/config/instruments.cfg`, the profile file, and restart the controller
GPIB instrument does not respond	Check cable, GPIB address, and instrument interface selection
RS-232 instrument does not respond	Check cable, port, baud rate, data bits, parity, and stop bits

TCP instrument does not respond	Check host address, port, and network connectivity
Instrument Read operation fails	Check query command and response-processing definitions
Macro does not appear	Verify its filename and location in `/macros`
Macro fails during execution	Verify instrument availability, capabilities, and macro instructions
Workspace values differ from instrument settings	Use Read to obtain actual values; loading a workspace does not send commands
Diagnostic files occupy excessive space	Download or delete obsolete archived logs and measurement files

8.11 Summary

The AK3Y Instrument Controller provides wireless access to laboratory instruments through a standard web browser.

Its network configuration can be changed without modifying the firmware, using a temporary wireless access point and a browser-based setup procedure.

Firmware updates can be installed through the controller's web interface using compatible ESP32 application images, eliminating the need for a direct USB connection during normal maintenance.

The About screen, OLED display, File Management facilities, and diagnostic logs provide information useful for identifying network, storage, firmware, and instrument communications problems.

When troubleshooting, it is important to distinguish between the controller's browser connection, communications with physical instruments, and the interpretation of instrument commands or macro instructions.

Together, these facilities allow the controller to be maintained and expanded as new instruments and measurement requirements are introduced.

Appendix A

Instrument Configuration File Reference

This appendix explains how to create and edit the instrument configuration file. It is intended for users of the controller, including those who install precompiled firmware and do not work with the software source code.

A.1 Purpose and file location

The file /config/instruments.cfg, stored on the controller's microSD card, lists the instruments available to the controller. Each instrument entry supplies a name, identifies its instrument profile, and specifies how the controller communicates with it.

The instrument configuration file does not contain the actual commands used to operate an instrument. Those commands are defined in separate instrument profile files, normally stored in /instruments/, and are described in Appendix B.

The controller can start with a newly formatted FAT32 microSD card even if instruments.cfg and the instrument profiles have not yet been installed. Add these files when you are ready to configure instruments.

A.2 Organization and format of the configuration file

Each instrument has its own section, beginning with a section identifier enclosed in square brackets. The lines that follow describe that instrument.

For example:

```
[MARCONI_2024]
name=Marconi 2024 Signal Generator
profile=marconi_2024.cfg
interface=GPIB
address=20
```

In this example, *MARCONI_2024* is the section identifier. It identifies this instrument within the configuration file. Use a different section identifier for each instrument.

Each remaining line contains a parameter name, an equals sign (=), and the parameter value. For example, *name=Marconi 2024 Signal Generator* specifies the descriptive name displayed for the instrument. The profile parameter identifies the instrument-specific profile file. The interface parameter selects the communications connection. Because this example uses GPIB, address=20 supplies the instrument's IEEE-488 address.

Additional instruments are defined by adding more sections to the same file.

Formatting rules

1. Each instrument begins with a section identifier in square brackets, such as [MARCONI_2024].
2. Write one parameter=value assignment per line. The first equals sign separates the parameter name from its value.
3. Blank lines are allowed and can be used to separate instrument entries.
4. Comments may be placed on separate lines beginning with # or ;. Spaces before these comment characters are permitted.
5. Spaces before or after a parameter name or value are ignored. For example, address = 20 is equivalent to address=20.
6. Capitalization matters for section identifiers, parameter names, and interface values. Use GPIB, RS232, or TCP exactly as shown. Parity values are not case-sensitive.
7. Use each parameter only once in a section. If it is repeated, the first value is used.
8. Give each instrument a unique section identifier. Avoid repeating section names.
9. Unrecognized parameter names are ignored. A misspelled required parameter therefore causes the instrument definition to be incomplete.
10. Do not add comments after parameter values. For example, *address=20 ; generator* is not a valid address entry.

A.3 Parameters required for every instrument

Every instrument section must include name, profile, and interface. There are no default values for these three parameters.

Parameter	What to enter	Purpose
name	Descriptive text (up to 79 characters)	Name used to identify the instrument in the controller.
profile	Profile filename (up to 95 characters)	Identifies the instrument profile containing its supported commands.
interface	GPIB, RS232, or TCP	Selects the communications interface. The value must be capitalized exactly as shown.

The section identifier itself may contain up to 31 characters. Use short, distinct identifiers that are easy to recognize.

A.4 GPIB instruments

For an instrument connected to the IEEE-488 bus, set interface=GPIB and include an address parameter.

Parameter	Accepted value	Meaning
address	Whole number from 0 through 30	Primary GPIB address of the instrument.

```
[SPECTRUM]
name=HP E4407B Spectrum Analyzer
profile=e4407b.cfg
interface=GPIB
address=18
```

The address must agree with the address set on the physical instrument. A valid address entry does not, by itself, confirm that the instrument is connected or responding.

A.5 RS-232 instruments

For an instrument connected to one of the controller's two serial ports, set interface=RS232 and supply all five serial communication parameters.

Parameter	Accepted value	Meaning
port	1 or 2	Controller RS-232 port.
baud	Whole number from 1 through 4,000,000	Requested serial data rate, in bits per second.
data_bits	5, 6, 7, or 8	Number of data bits.
parity	none, even, or odd	Parity setting; capitalization is not significant.
stop_bits	1 or 2	Number of stop bits.

```
[MARCONI_SERIAL]
name=Marconi 2024-Serial
profile=marconi_2024_rs232.cfg
interface=RS232
port=1
baud=9600
data_bits=8
parity=none
stop_bits=1
```

The serial settings must match those configured on the instrument. Although the configuration file accepts baud rates within the range shown, successful communication depends on the capabilities of the instrument and the physical serial connection.

A.6 TCP network instruments

For an instrument reached over a TCP network connection, set interface=TCP and specify its network host and TCP port.

Parameter	Accepted value	Meaning
host	IP address or host name (up to 63 characters)	Network destination of the instrument.
port	Whole number from 1 through 65535	TCP port used by the instrument.

```
[LANSIM]
name=LAN Instrument Simulator
profile=lan_simulator.cfg
interface=TCP
host=192.168.1.155
port=5025
```

The controller accepts the host text as entered. An apparently valid entry does not guarantee that the host exists, that its name can be resolved, or that its TCP port is accessible.

A.7 Complete configuration example

The following example defines four instruments: two GPIB instruments, one RS-232 instrument, and one TCP instrument. The network address and serial settings are illustrative; substitute the settings for your own equipment.

```
# Instrument definitions for the AK3Y Controller

[RFGEN]
name=RF Generator - GPIB
profile=marconi_2024.cfg
interface=GPIB
address=20

[RFGEN_SERIAL]
name=RF Generator – RS232
profile=marconi_2024_rs232.cfg
interface=RS232
port=1
baud=9600
data_bits=8
parity=none
stop_bits=1

[SPECTRUM]
name=HP E4407B Spectrum Analyzer
profile=e4407b.cfg
interface=GPIB
address=18

[LANSIM]
name=LAN Instrument Simulator
profile=lan_simulator.cfg
interface=TCP
host=192.168.1.155
port=5025
```

A.8 Installing and checking the file

Using the controller's File Management page, upload instruments.cfg to the /config directory. Upload each corresponding instrument profile file to the /instruments directory. Restart the controller after changing the instrument list, then open the Instruments page and verify that the expected instruments can be selected.

If an instrument is not available, check the file location and spelling, confirm that all required parameters are present, and verify that the interface-specific settings are valid. Also check that the named profile file has been uploaded. The controller can start normally even when no instruments have been configured.

A.9 Configuration file limitations

The following details are primarily relevant when troubleshooting a configuration file:

- Instrument section identifiers are limited to 31 characters. Instrument names are limited to 79 characters, profile values to 95 characters, and TCP host values to 63 characters.
- Keep configuration lines short. The file reader uses a 160-byte line buffer, so lines should remain below that limit, including any line-ending characters.
- Numerical parameters must contain only a valid whole number, with no explanatory text or inline comment.
- If a required parameter is absent, a value is too long, or a parameter is outside its accepted range, that instrument definition cannot be loaded.
- Unknown parameter names are ignored rather than reported as errors. Check spelling carefully.
- Duplicate section identifiers should not be used; repeated sections may not be interpreted as intended.

A.10 Alphabetical parameter reference

Parameter	Used with	See section
address	GPIB	A.4
baud	RS232	A.5
data_bits	RS232	A.5
host	TCP	A.6
interface	All	A.3
name	All	A.3
parity	RS232	A.5
port	RS232 and TCP	A.5, A.6
profile	All	A.3
stop_bits	RS232	A.5

Appendix B
Instrument Profile File Reference

B.1 Introduction

An instrument profile is a plain-text .cfg file which specifies all of the commands and control functions desired for the device. Each instrument must have at least one such profile in order to be recognized by the Controller and to allow those functions to be displayed within the browser's Control Panel. Multiple profiles can be created for a single device, with each profile including commands appropriate for a specific instrument function.

The /config/instruments.cfg equipment inventory file (see Appendix A) provides the pointers to these instrument profiles, associating each with a specific communications interface (i.e., GPIB, RS-232 or TCP), address and/or port.

B.2 A Simple Instrument Profile: Marconi 2024

The easiest way to understand the construction of an instrument profile is through examples. Let's begin with a simple profile for an RF signal generator, the Marconi 2024.

The Marconi 2024 was manufactured from approximately 1994 to 2014. It provides fixed-frequency and swept-frequency operation over the range of 9 kHz to 2.4 GHz and can be controlled through either a GPIB or RS-232 interface. These instruments are still readily available on the surplus market, often for around $200.

Our first example defines only two controls: **Frequency** and **RF Level**. These allow the operator to set the output frequency and power level and read their current values. The complete profile is shown below.

```
[instrument]
manufacturer=Marconi
model=2024
description=RF Signal Generator
default_function=Basic RF Generator
default_enter=CFRQ:MODE FIXED
ready_query=*OPC?
ready_response=1
ready_timeout_ms=30000

[frequency]
type=number
quantity=frequency
label=Frequency
base_unit=Hz
set=CFRQ:VALUE {value}
query=CFRQ?
response_prefix=:CFRQ:VALUE
response_terminator=;
minimum=9000
maximum=2400000000
```

```
[level]
type=number
quantity=power_level
label=RF Level
base_unit=dBm
set=RFLV:VALUE {value}DBM
query=RFLV?
response_prefix=;VALUE
response_terminator=;
minimum=-140
maximum=13
```

B.2.1 The [instrument] Section

Every instrument profile must begin with an `[instrument]` section. This section identifies the instrument and specifies certain commands that apply to the instrument as a whole, rather than to any particular control.

The first three parameters are straightforward:

- `manufacturer=Marconi` identifies the instrument manufacturer.
- `model=2024` identifies the instrument model.
- `description=RF Signal Generator` provides a brief description of the instrument.

These parameters contain descriptive text, not commands sent to the instrument. They help identify the instrument but do not determine which controls appear in its Control Panel.

The next parameter is:

```
default_function=Basic RF Generator
```

Every instrument Control Panel has a **Function selector**. The first selection is the instrument's basic function, which contains its ordinary controls. Unless otherwise specified, this function is called *Basic Controls*.

The `default_function` parameter allows us to give these basic controls a more descriptive name. In this example, the Function selector displays **Basic RF Generator**.

The word *Basic* has no special significance to the controller. We could just as easily have called this function *Fixed Frequency Operation* or *Normal Operation*. The parameter simply supplies the label displayed in the Function selector.

For this simple profile, the basic function contains both controls defined in the file: Frequency and RF Level. Later, when we introduce additional functions using `type=group`, we will see how the controller determines which controls become associated with this basic instrument function.

The next parameter is:

```
default_enter=CFRQ:MODE FIXED
```

This is an actual command sent to the Marconi 2024 when the basic function is entered. It places the instrument in fixed-frequency mode, rather than swept-frequency mode. The command is specific to the Marconi 2024. Other instruments may use different commands, or may not require an entry command at all. The `default_enter` parameter is optional.

The remaining three parameters define an instrument readiness check:

```
ready_query=*OPC?
ready_response=1
ready_timeout_ms=30000
```

The `ready_query` parameter specifies the command used to determine whether the instrument has completed an operation. The Marconi supports the standard `*OPC?` query, which returns `1` when the operation is complete.

The `ready_response` parameter specifies the expected reply. In this example, the controller expects the character `1`.

The `ready_timeout_ms` parameter specifies the maximum time allowed for the query and its response, in milliseconds. A value of 30000 corresponds to 30 seconds. The controller permits values from 1 to 120000 milliseconds.

These three parameters are used when the controller executes a `WAIT instrument ready` command, which is particularly useful in automated measurements. The controller sends the query once and checks the returned response; it does not repeatedly poll the instrument.

These parameters are not needed simply to set or read the frequency or RF level.

B.2.2 The [frequency] Section

The next section defines the first operator control: `[frequency]`

Note that the text within square brackets is the **capability name**. It identifies this particular instrument function within the profile and is also used when referring to the capability from controller commands and macros.

The first parameter in this subsection is:

```
type=number
```

This tells the controller to create a numerical control, consisting of a value-entry field and the appropriate Set and/or Read buttons in the graphical interface.

The supported control types include `number`, `select`, `action`, and `toggle_action`. A fifth type, `group`, is used to organize controls into additional functions. We will examine these types in subsequent sections.

The next three parameters describe the numerical quantity:

```
quantity=frequency
label=Frequency
base_unit=Hz
```

The `quantity` parameter identifies the physical quantity being controlled. Here it is `frequency`.

The `label` parameter specifies the text displayed above the entry field in the browser. Thus, the operator sees **Frequency**, rather than the internal capability name `[frequency]`.

The `base_unit` parameter specifies the unit used internally for numerical values and in commands sent to the instrument. For frequency, the required base unit is `Hz`.

The controller recognizes Hz, kHz, MHz, and GHz when entering frequency values. For example, the operator may enter `796.009 MHz`, which the controller converts to 796009000 Hz before sending the command.

The next two parameters define how frequency is set and read:

```
set=CFRQ:VALUE {value}
query=CFRQ?
```

The `set` parameter contains the instrument command used to change the frequency. The special expression `{value}` is a placeholder for the numerical value entered by the operator.

For example, entering `796.009 MHz` and pressing Set produces the command:

```
CFRQ:VALUE 796009000
```

The `query` parameter specifies the command used to read the current frequency. When the operator presses Read, the controller sends `CFRQ?` to the instrument.

These commands are taken directly from the Marconi programming language. They are not universal controller commands.

The next two parameters describe how to interpret the instrument's response:

```
response_prefix=:CFRQ:VALUE
response_terminator=;
```

Some instruments return more than a simple numerical value. For example, a frequency query might return:

```
:CFRQ:VALUE 796009000;MODE FIXED;
```

The `response_prefix` tells the controller to locate the text `:CFRQ:VALUE` and begin extracting the value immediately afterward.

The `response_terminator` tells it to stop at the first semicolon following that prefix.

The resulting numerical value is `796009000`, which the controller can display as `796.009 MHz`.

Finally, the frequency section specifies:

```
minimum=9000
maximum=2400000000
```

These parameters define the permissible range of values for this control, expressed in the declared base unit. In this case, the range is 9000 Hz to 2400000000 Hz, corresponding to 9 kHz through 2.4 GHz. The controller displays these limits beneath the Frequency entry field and checks entered values against them before issuing a Set command.

B.2.3 The [level] Section

The final section defines the RF output level:

```
[level]
type=number
quantity=power_level
label=RF Level
base_unit=dBm
set=RFLV:VALUE {value}DBM
query=RFLV?
response_prefix=;VALUE
response_terminator=;
minimum=-140
maximum=13
```

This section follows exactly the same general structure as `[frequency]`.

The `type=number` parameter again specifies a numerical control. The `label=RF Level` parameter determines the text displayed in the browser.

However, the physical quantity is now `power_level`, and the required base unit is `dBm`. Unlike frequency, for which several engineering units are recognized, `power_level` uses dBm directly. The controller does not automatically convert between dBm and watts.

The Set command is:

```
set=RFLV:VALUE {value}DBM
```

For example, entering `-20 dBm` causes the controller to send:

```
RFLV:VALUE -20DBM
```

Notice that `DBM` follows the `{value}` placeholder without an intervening space. Again, this is part of the command syntax required by the Marconi.

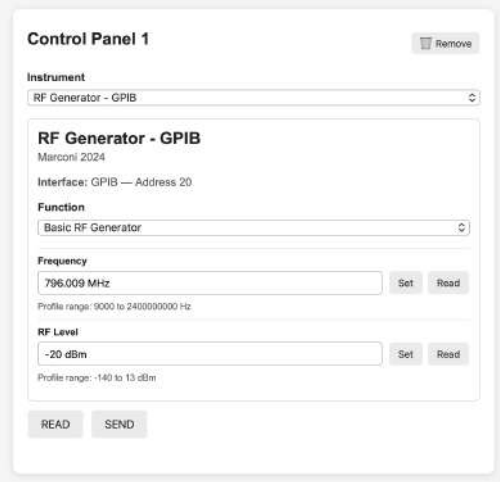
The `query=RFLV?` parameter defines the command used to read the current output level.

As before, the `response_prefix` and `response_terminator` parameters extract the numerical value from the returned text. In this case, the prefix is `;VALUE ` , including the trailing space.

Finally, `minimum=-140` and `maximum=13` establish the permissible RF level range, from −140 dBm to +13 dBm.

B.2.4 What the Controller Displays

When this profile is used to create a Control Panel, the Function selector displays **Basic RF Generator**, with two numerical controls beneath it: Frequency and RF Level.



Each control has its own Set and Read buttons, because both a `set` and a `query` command are defined. No additional functions appear because this simple profile contains no `type=group` sections (see below).

This example illustrates the fundamental principle of instrument profiles. T**he controller creates the browser controls from the profile definitions, while the actual commands sent to the instrument come from its programming manual.**

In the following sections, we will expand the Marconi profile to include swept-frequency operation and examine the additional control types needed for more complex instruments.

B.3 Adding a Frequency Sweep Function

Our first example provided only two controls for the Marconi 2024: Frequency and RF Level. These appeared under the **Basic RF Generator** selection in the Control Panel's Function menu.

The Marconi 2024 also supports frequency sweeping, in which the output frequency changes between specified starting and stopping frequencies, and steps either linearly or logarithmically. We would like to provide a second set of controls for configuring and operating this feature, without cluttering the basic Control Panel.

The controller accomplishes this using a new capability type, `type=group`.

B.3.1 Defining a Function Group

The following section is added immediately after the `[level]` section of our original profile:

```
[sweep]
type=group
label=Carrier Frequency Sweep
enter=CFRQ:MODE SWEPT
enter_before_action=true
members=sweep_start,sweep_stop,sweep_step,sweep_log_step,sweep_time,sweep_mode,sweep_type,swe
ep_trigger,sweep_start_action,sweep_reset_action
```

The first line,`[sweep]`, identifies the new function within the profile. As with `[frequency]` and `[level]`, this is an internal identifier, not necessarily the text displayed in the browser.

The parameter `type=group` tells the controller that this section defines a new parameterized function rather than an individual control. The controller adds another choice to its Function selector.

The next parameter, `label=Carrier Frequency Sweep` specifies the text displayed for this choice. Thus, the Function selector now offers two selections: **Basic RF Generator** and **Carrier Frequency Sweep**.

The parameter

```
enter=CFRQ:MODE SWEPT
```

specifies a command to be sent to the Marconi when the sweep function is entered. This changes the generator from fixed-frequency operation to swept-frequency operation.

Recall that our original `[instrument]` section contained:

```
default_enter=CFRQ:MODE FIXED
```

Thus, the two functions have their own entry commands. Selecting Basic RF Generator can place the instrument in fixed-frequency mode, while selecting Carrier Frequency Sweep can place it in swept-frequency mode.

The parameter

```
enter_before_action=true
```

provides an additional safeguard when executing actions associated with the sweep function. It requests that the function's entry command be executed before the associated action, ensuring that the generator is placed in swept-frequency mode *before* the sweep command is issued.

Finally, the `members` parameter identifies the individual controls belonging to the sweep function. These names correspond to section identifiers elsewhere in the profile.

For example, `sweep_start` refers to the `[sweep_start]` section, while `sweep_type` refers to `[sweep_type]`, etc.

Note: The `members` parameter does not contain commands sent to the instrument. Rather, it is a list of internal capability identifiers that tells the controller which controls to display when Carrier Frequency Sweep is selected. Also, all entries belong on a single line.

The basic function does not need a `members` parameter. It is created automatically, and its controls are those not assigned to a separate function group. The first `type=group` section marks the beginning of the additional function definitions.

B.3.2 Numerical Sweep Controls

The first two sweep controls specify the starting and stopping frequencies. For example, the following lines are concatenated below the `[sweep]` function definition:

```
[sweep_start]
type=number
quantity=frequency
label=Start Frequency
base_unit=Hz
set=SWEEP:CFRQ:START {value}
query=SWEEP:CFRQ?
response_prefix=:SWEEP:CFRQ:START
response_terminator=;
minimum=9000
maximum=2400000000
```

This definition is almost identical in structure to the `[frequency]` section discussed earlier. It creates a numerical entry field labeled Start Frequency, with Set and Read buttons. The important difference is the instrument command. Instead of changing the generator's fixed output frequency, the command `SWEEP:CFRQ:START` establishes the starting frequency for a sweep.

The `[sweep_stop]` section operates similarly, using the command `SWEEP:CFRQ:STOP`.

Note that the sweep profile also includes controls for step size and step time. These demonstrate that a function group may contain several numerical controls, each with its own physical quantity, units, limits, and instrument commands.

B.3.3 Selection Controls: `type=select`

Not all instrument settings are numerical. Some may require the operator to choose among a limited number of operating modes.

For example, the Marconi provides single and continuous sweeping. The corresponding profile section is:

```
[sweep_mode]
type=select
label=Sweep Mode
set=SWEEP:MODE {value}
query=SWEEP?
response_prefix=:SWEEP:MODE
response_terminator=;
options=Single:SNGL,Continuous:CONT
```

The parameter `type=select` tells the controller to create a selection list rather than a numerical entry field.

The `options` parameter defines the available choices. Each choice consists of two parts separated by a colon in the format

```
options=Displayed text:Instrument value
```

The individual choices are separated by commas. In this example, the operator sees **Single** and **Continuous**. Selecting **Single** supplies the value `SNGL`, while selecting **Continuous** supplies `CONT`. Thus, selecting Continuous and pressing Set produces `SWEEP:MODE CONT`.

The command values must correspond to those recognized by the instrument. The controller does not translate them into another manufacturer's command language.

Another selection control, `[sweep_type]`, provides the choices Linear and Logarithmic:

```
options=Linear:LIN,Logarithmic:LOG
```

These choices are particularly useful because they can also determine which other controls are displayed (see B.3.4 below).

B.3.4 Conditional Controls: `visible_when`

In the Marconi 2024, a linear sweep requires an incremental frequency to be supplied, whereas a logarithmic sweep requires a percentage change. Rather than displaying both controls at all times, the Marconi profile uses the `visible_when` parameter.

The linear step-size control contains:

```
[sweep_step]
type=number
quantity=frequency
label=Step Size
base_unit=Hz
set=SWEEP:CFRQ:INC {value}
query=SWEEP:CFRQ?
response_prefix=;INC
response_terminator=;
minimum=1
maximum=2400000000
visible_when=sweep_type:LIN
```

The final parameter means that the label `Step Size` is displayed when the capability named `sweep_type` has the value `LIN`.

Similarly, the logarithmic step-size control contains:

```
[sweep_log_step]
type=number
quantity=percent
label=Log Step
base_unit=%
set=SWEEP:CFRQ:LOGINC {value}
query=SWEEP:CFRQ?
response_prefix=;LOGINC
response_terminator=;
minimum=0.01
maximum=50
visible_when=sweep_type:LOG
```

Here, `visible_when=sweep_type:LOG` causes the **Log Step** control to appear when Logarithmic is selected.

Notice that `visible_when` refers to the internal capability name `[sweep_type]` and the instrument value `LIN` or `LOG`, not the displayed text Linear or Logarithmic.

This feature allows the Control Panel to present only those controls relevant to the selected operating mode.

B.3.5 Response Units

The sweep-time control function below introduces other useful parameters:

```
[sweep_time]
type=number
quantity=time
label=Step Time
base_unit=s
set=SWEEP:CFRQ:TIME {value}S
query=SWEEP:CFRQ?
response_prefix=;TIME
response_unit=ms
minimum=0.05
maximum=10
```

The `base_unit=s` parameter specifies that time values are represented internally in seconds. The controller accepts supported engineering units, including milliseconds.

However, the Marconi reports sweep time in milliseconds. Thus, the parameter

```
response_unit=ms
```

tells the controller how to interpret the numerical value extracted from the instrument's response. For example, a returned value of 250 represents 250 milliseconds, or 0.25 seconds.

Note that this is different from `response_terminator`, which identifies where a value ends in a response string. The `response_unit` parameter identifies the physical units of that value.

B.3.6 Action Buttons: `type=action`

Some instrument operations require only a command, rather than an entered value.

For example, the Marconi provides a command to reset the sweep:

```
[sweep_reset_action]
type=action
label=Reset Sweep
action=SWEEP:RESET
resets_toggle=sweep_start_action
```

The parameter `type=action` creates a graphical interface button labeled Reset Sweep. Pressing the button sends the command specified by `action`, in this case `SWEEP:RESET`.

Unlike `type=number` or `type=select`, an action control does not require a value-entry field or an `options` list.

The additional parameter `resets_toggle` is discussed in the next section.

B.3.7 Two-State Action Buttons: `type=toggle_action`

Some operations have two complementary actions, such as starting and stopping a sweep.

Our Marconi profile defines:

```
[sweep_start_action]
type=toggle_action
label=Start Sweep
label_on=Stop Sweep
action=SWEEP:GO
action_on=SWEEP:HALT
```

The parameter `type=toggle_action` creates a two-state button.

Initially, the button is labeled **Start Sweep**. Pressing it sends `SWEEP:GO`, and the button changes to **Stop Sweep**.

Pressing it again sends `SWEEP:HALT`, returning the button to its original state.

The `label` and `label_on` parameters specify the two button labels, while `action` and `action_on` specify the corresponding instrument commands.

Returning to the `[sweep_reset_action] control, we had the command:`

```
resets_toggle=sweep_start_action
```

This tells the browser that pressing Reset Sweep should also reset the displayed state of the toggle button identified by `[sweep_start_action]`.

It is important to recognize that the toggle button represents the controller's current user-interface state. It is not necessarily a direct measurement of the instrument's actual operating state, particularly if the instrument has been controlled externally.

B.3.8 Hidden Capabilities: `ui=false`

The Marconi profile includes one additional capability that is not intended to appear as a normal Control Panel control:

```
[frequency_mode]
```

```
type=select
ui=false
label=Frequency Mode
set=CFRQ:MODE {value}
query=CFRQ?
response_prefix=;MODE
response_terminator=;
options=Fixed:FIXED,Swept:SWEPT
```

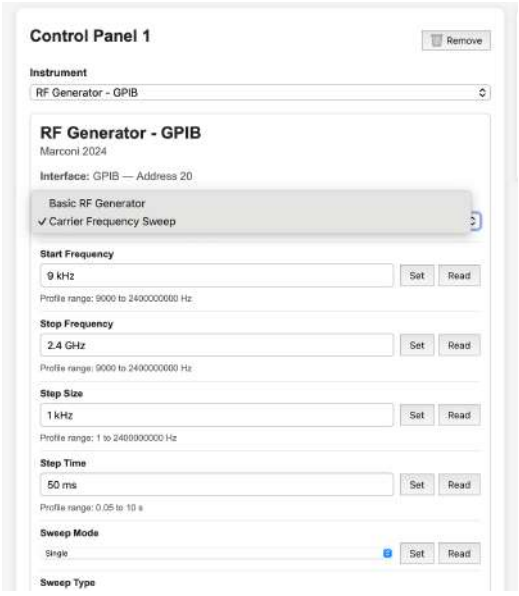
The parameter `ui=false` tells the controller not to display this capability as an ordinary browser control. Nevertheless, the capability remains defined and can be referenced by controller commands and macros.

This is useful when an instrument setting must be available for automation but does not need to occupy space in the graphical Control Panel.

In this example, the fixed and swept frequency modes are already selected through the Function selector and its associated entry commands. A separate Frequency Mode selection control would therefore be largely redundant.

B.3.9 The Completed Sweep Function

With these additions, the Marconi Control Panel provides two distinct operating functions (see partial Control Panel below).



Basic RF Generator contains the Frequency and RF Level controls described in Section B.2. **Carrier Frequency Sweep** contains controls for starting and stopping frequencies, linear or logarithmic step size, sweep timing, sweep mode, sweep type, sweep triggering, and sweep start, stop, and reset operations.

The Function selector allows the operator to move between these two sets of controls without creating a separate instrument profile.

The mechanisms described above are sufficient to describe a wide variety of instruments, even though the actual command strings and available settings will differ from one manufacturer to another.

B.4 Instrument Profile Reference Tables

Table B-1. Supported capability types

type value	Purpose	Principal parameters
number	Numerical input and readback	quantity, base_unit, set, query, minimum, maximum
select	Selection among predefined choices	options, set, query
action	Button that sends a command	label, action
toggle_action	Two-state command button	label, label_on, action, action_on
group	Adds another Function selection	label, members, enter, enter_before_action

Table B-2. Instrument profile parameters

Parameter	Permissible value / syntax	Purpose
manufacturer	Text	Manufacturer identification
model	Text	Model identification
description	Text	Instrument description
default_function	Text	Basic function label (default: Basic Controls)
default_enter	Instrument command	Command associated with entering basic function
ready_query	Instrument query	Readiness check command
ready_response	Response text	Expected readiness response
ready_timeout_ms	Integer: 1–120000	Readiness timeout (ms)
type	number, select, action, toggle_action, group	Capability type (Table B-1)
label	Text	Displayed control or group label
quantity	One of Table B-3 quantities	Numerical quantity
base_unit	Matching unit from Table B-3	Base unit for numerical values
set	Command with {value}	Sets an instrument parameter
query	Instrument query	Reads an instrument parameter
response_prefix	Text (may include spaces)	Beginning marker for returned value
response_terminator	Text; may be empty	Ending marker for returned value
response_unit	Compatible engineering unit	Units of numerical response
minimum	Number in base units	Minimum permitted numerical value
maximum	Number in base units	Maximum permitted numerical value
options	Label:Value,Label:Value,…	Choices for select control
visible_when	capability:value	Conditional visibility
ui	true or false	Show/hide normal browser control
enter	Instrument command	Command associated with group entry
enter_before_action	true or false	Requests group entry command before action
members	Comma-separated capability IDs	Controls in a function group
action	Instrument command	Action-button command
label_on	Text	Alternate toggle label

action_on	Instrument command	Alternate toggle command
resets_toggle	Capability identifier	Resets named toggle's displayed state

Table B-3. Supported numerical quantities and units

quantity	Required base_unit	Accepted input units
frequency	Hz	Hz, kHz, MHz, GHz
time	s	ps, ns, us, ms, s
voltage	V	uV, mV, V, kV
current	A	uA, mA, A
power	W	uW, mW, W, kW
power_level	dBm	dBm
attenuation	dB	dB
amplitude_scale	dB	dB
count	count	count
percent	%	%
resistance	ohm	ohm, kohm, Mohm

Bare numbers are interpreted in the declared base unit. Frequency suffixes are case-insensitive; other suffixes are case-sensitive except for an uppercase final S in time units.

Table B-4. RS-232 and TCP termination parameters

Parameter	Permissible values	Purpose
serial_command_terminator	NONE, CR, LF, CRLF	Characters appended to RS-232 commands
serial_response_terminator	CR, LF, CRLF	Characters that end an RS-232 response
tcp_command_terminator	NONE, CR, LF, CRLF	Characters appended to TCP commands
tcp_response_terminator	CR, LF, CRLF	Characters that end a TCP response

NONE is permitted for command termination, but not for response termination. CR = carriage return; LF = line feed; CRLF = carriage return followed by line feed. These settings are defined in the `[instrument]` section and are not used for GPIB.

Appendix C
Macro Command Language Reference

C.1 Introduction

Chapter 5 introduced the macro facility of the AK3Y Instrument Controller, using two examples: FrequencySweep and FindSignal. This appendix provides a more detailed description of the macro command language, including file organization, parameter definitions, command syntax, variables, numerical operations, and programming restrictions.

Macros provide a means of automating instrument operations without modifying the controller firmware. A macro can set instrument parameters, initiate operations, retrieve measurements, execute repeated sequences, and record results.

An important feature of the macro language is its independence from the instrument communications interface. A macro refers to an instrument by its identifier in the instrument inventory and to a particular capability by its name in the instrument profile. The controller determines the appropriate GPIB, RS-232, or TCP command from these definitions.

For example:

```
SET RFGEN frequency 146MHz
```

sets the frequency of the instrument identified as `RFGEN`, using the `frequency` capability defined in its instrument profile.

The macro does not need to know whether the instrument is connected through GPIB, RS-232 or via Wifi, nor the particular command syntax required by its manufacturer.

The macro language contains two general classes of instructions:

> **Instrument commands** communicate with connected instruments. These include `SET`, `QUERY`, `ACTION`, `ACTIONON`, `ENTER`, and `WAIT`.
>
> **Macro control commands** control the execution sequence or record information. These include `DELAY`, `SWEEP`, `ENDSWEEP`, and `LOG`.

The following sections describe the organization of macro files and the syntax of these instructions.

C.2 Macro File Organization

Macros are ordinary text files with the extension `.cfg` stored in the `/macros` directory of the controller's microSD card.

A macro consists of named sections, much like the instrument profiles described in Appendix B.

A typical macro has the following organization:

```
[macro]
description=Example measurement procedure

[parameter:frequency]
type=number
quantity=frequency
label=Test Frequency
base_unit=Hz
default=100MHz

[result:MeasuredFrequency]
label=Measured Frequency
source=FreqMeasured
unit=Hz

[steps]
1=SET RFGEN frequency {frequency}
2=WAIT RFGEN ready
3=QUERY SPECTRUM marker_frequency AS FreqMeasured
```

Four section types are recognized:

- `[macro]` contains descriptive information.
- `[parameter:identifier]` defines an operator input.
- `[result:identifier]` defines a measurement result to be displayed.
- `[steps]` contains the executable instructions.

The `[macro]` section is mandatory, as is at least one executable step. Input parameters and displayed results are optional.

C.2.1 The [macro] Section

The `[macro]` section identifies the purpose of the macro.

For example:

```
[macro]
description=Find the strongest signal within a specified frequency range
```

The recognized parameters are summarized below.

Parameter	Purpose	Example
`description`	Describes the purpose of the macro	`description=Find the strongest signal`

`name`	Legacy parameter, accepted but ignored	`name=Find Signal`

The filename determines the macro name displayed in the browser.

Thus, the file `FindSignal.cfg` appears in the Macro selector as **FindSignal**, while `FrequencySweep.cfg` appears as **FrequencySweep**.

Although the parser accepts a `name=` entry for compatibility with older macro files, its value is ignored. The filename, without its `.cfg` extension, is the authoritative macro name.

C.2.2 Input Parameters

Input parameters allow the operator to specify measurement conditions before executing a macro.

Each parameter is defined by a section heading of the form:

```
[parameter:identifier]
```

For example:

```
[parameter:start_frequency]
type=number
quantity=frequency
label=Start Frequency
base_unit=Hz
default=147MHz
```

The identifier `start_frequency` is the internal parameter name. It is used to reference the parameter within macro instructions.

The `label` parameter specifies the text displayed in the Macro Panel.

The `default` parameter specifies the initial value presented to the operator.

In this example, the Macro Panel displays a numerical input labeled Start Frequency, initially set to 147 MHz.

The value can be referenced in an instruction by enclosing its identifier in braces:

```
{start_frequency}
```

For example:

```
SET RFGEN frequency {start_frequency}
```

When the macro executes, the controller substitutes the operator's selected value for `{start_frequency}`.

The supported input parameter types are described in Section C.3.

C.2.3 Displayed Results

A macro may define one or more measurement results to be displayed in the Macro Panel.

Each result uses a section heading of the form:

```
[result:identifier]
```

For example:

```
[result:PeakFrequency]
label=Peak Frequency
source=PeakFreq
unit=Hz
```

The recognized parameters are:

Parameter	Purpose	Required
`label`	Text displayed beside the result	Yes
`source`	Runtime variable containing the result	Yes
`unit`	Unit displayed with the result	No

The `source` parameter identifies a variable populated during macro execution.

For example:

```
QUERY SPECTRUM marker_frequency AS PeakFreq
```

reads the spectrum analyzer's marker frequency and stores the returned value in the variable `PeakFreq`.

The result definition then associates that variable with the displayed label **Peak Frequency**.

The `unit` parameter is display information only. It does not convert the value between engineering units. Consequently, the value stored in the source variable must already be expressed in the appropriate units.

Result identifiers and source variable names must begin with a letter or underscore and may contain letters, digits, and underscores. Duplicate result identifiers are not permitted.

C.2.4 The [steps] Section

The `[steps]` section contains the instructions executed by the controller.

For example:

```
[steps]
1=SET RFGEN frequency 146MHz
2=WAIT RFGEN ready
3=DELAY 1000
4=QUERY SPECTRUM marker_frequency AS MeasuredFreq
```

Each instruction consists of a step number, an equals sign, and a command.

Step numbers must begin with 1 and increase consecutively by 1.

Missing numbers, duplicate numbers, or numbering that begins with a value other than 1 cause the macro to be rejected.

The current implementation permits a maximum of 32 instructions per macro. Each instruction may contain up to 223 characters, including its arguments but excluding the step number and equals sign.

Instructions normally execute in numerical order. The `SWEEP` and `ENDSWEEP` commands provide the ability to repeat a sequence of instructions.

C.3 Macro Input Parameter Types

The macro configuration parser recognizes three input parameter types: `number`, `boolean`, and `select`. These types determine how an operator enters a value and how that value is interpreted when substituted into macro instructions.

C.3.1 Numerical Parameters: type=number

A numerical parameter allows the operator to enter a value, optionally accompanied by an engineering prefix and unit.

For example:

```
[parameter:frequency]
type=number
quantity=frequency
label=Frequency
base_unit=Hz
default=100MHz
```

For a numerical parameter, the following definitions are required:

- `type=number`
- `label`
- `quantity`
- `base_unit`

The `default` parameter is optional in the file format, but a value must be available when the parameter is actually used.

The numerical value is converted into the specified base unit before being substituted into a macro instruction. For example, the input `100MHz` becomes `100000000` when the base unit is Hz.

The macro parser recognizes the following engineering prefixes:

Prefix	Multiplier	Example
`m`	0.001	`250ms`
`k` or `K`	1,000	`100kHz`
`M`	1,000,000	`146MHz`
`G`	1,000,000,000	`2GHz`

The prefixes are case-sensitive where necessary. In particular, `m` means milli, whereas `M` means mega.

If a unit follows an engineering prefix, it must match the declared `base_unit` exactly. For example, with `base_unit=Hz`, the values `146MHz` and `146M` are accepted. The parser also accepts a number without a suffix, treating it as a value already expressed in base units.

An important restriction is that the macro parser does not implement every engineering-unit conversion supported by instrument Control Panels. In particular, the macro parameter parser recognizes only the prefixes listed above. For example, it does not recognize the micro prefix `u` in a numerical macro input.

C.3.2 Boolean Parameters: type=boolean

A boolean parameter represents a condition with two possible values, such as enabled or disabled.

For example:

```
[parameter:output_enable]
type=boolean
label=Enable Output
default=On
```

The parser accepts the following values:

Input	Normalized value
`on`, `true`, `1`	`1`
`off`, `false`, `0`	`0`

Letter case is ignored for the textual values. Thus, `ON`, `On`, and `on` are equivalent.

When the parameter is substituted into a macro instruction, the normalized value is `1` or `0`.

For example:

```
SET RFGEN output {output_enable}
```

would substitute `1` when the parameter is enabled.

The corresponding instrument capability must, of course, accept the value supplied by the macro.

Boolean parameters do not require `quantity` or `base_unit`.

C.3.3 Selection Parameters: type=select

A selection parameter allows the operator to choose among predefined values.

For example:

```
[parameter:sweep_mode]
type=select
label=Sweep Mode
options=Single:SNGL,Continuous:CONT
default=Single
```

The `options` parameter defines the available choices. Each option consists of two parts separated by a colon:

```
Displayed label:Value
```

Individual options are separated by commas.

In this example, the operator sees Single and Continuous. The values substituted into macro instructions are `SNGL` and `CONT`, respectively.

The default may specify either the displayed label or the corresponding value.

For example, both of the following are accepted:

```
default=Single
default=SNGL
```

The displayed label is matched without regard to case, while the corresponding option value is matched exactly.

A selection parameter requires `type`, `label`, and `options`.

C.3.4 Input Parameter Summary

Parameter	**number**	**boolean**	**select**
`type`	Required	Required	Required
`label`	Required	Required	Required

`quantity`	Required	Not required	Not required
`base_unit`	Required	Not required	Not required
`default`	Optional*	Optional*	Optional*
`options`	Not required	Not required	Required

- *Although the parser does not require a default, the parameter must have a valid value, supplied either by its default or by an operator override, when it is referenced during execution.*

A maximum of eight input parameters may be defined in a macro.

C.4 Instrument Commands

Instrument commands identify an instrument and a capability defined in that instrument's profile.

For example:

```
SET RFGEN frequency 146MHz
```

Here, `RFGEN` is the instrument identifier from `/config/instruments.cfg`, while `frequency` is the capability identifier from the associated instrument profile.

The controller uses these identifiers to determine the appropriate command and communications interface.

C.4.1 SET — Set an Instrument Parameter

Syntax:

```
SET instrument capability value
```

Example:

```
SET RFGEN frequency 146MHz
```

The `SET` command changes an instrument parameter.

The corresponding instrument profile must contain a `set=` command definition for the specified capability.

For the Marconi 2024, the frequency capability contains:

```
set=CFRQ:VALUE {value}
```

The controller substitutes the requested value and transmits the resulting command.

The value may be a literal number or a macro parameter or runtime variable enclosed in braces. For example:

```
SET RFGEN frequency {Freq_IN}
```

sets the signal generator frequency to the current value of the macro variable `Freq_IN`.

C.4.2 QUERY — Read an Instrument Parameter

Syntax:

```
QUERY instrument capability
```

or:

```
QUERY instrument capability AS variable
```

Examples:

```
QUERY SPECTRUM marker_frequency
QUERY SPECTRUM marker_frequency AS PeakFreq
```

The `QUERY` command reads an instrument parameter using the `query=` definition in its profile.

The optional `AS` clause assigns the returned value to a runtime variable. For example:

```
QUERY SPECTRUM marker_amplitude AS PeakLevel
```

reads the analyzer's marker amplitude and stores the result in `PeakLevel`.

The variable may subsequently be used by other macro instructions, displayed as a result, or recorded in a CSV file.

C.4.3 ACTION — Execute an Instrument Action

Syntax:

```
ACTION instrument capability
```

Example:

```
ACTION SPECTRUM peak_search
```

The `ACTION` command executes the command defined by the capability's `action=` parameter. Unlike `SET`, it does not require a numerical value.

Typical applications include instrument preset, starting a sweep, peak searching, and marker positioning.

C.4.4 ACTIONON — Execute an Alternate Action

Syntax:

```
ACTIONON instrument capability
```

The `ACTIONON` command executes the command defined by the capability's `action_on=` parameter. This is useful for capabilities defined as `type=toggle_action`.

For example, the Marconi sweep profile contains:

```
[sweep_start_action]
type=toggle_action
label=Start Sweep
label_on=Stop Sweep
action=SWEEP:GO
action_on=SWEEP:HALT
```

The corresponding macro commands are:

```
ACTION RFGEN sweep_start_action
ACTIONON RFGEN sweep_start_action
```

The first executes the normal action, starting the sweep. The second executes the alternate action, stopping it.

C.4.5 ENTER — Enter an Instrument Function

Syntax:

```
ENTER instrument function
```

The `ENTER` command invokes the entry behavior associated with a function defined in an instrument profile.

For example, the Marconi profile contains the following sweep group:

```
[sweep]
type=group
label=Carrier Frequency Sweep
enter=CFRQ:MODE SWEPT
```

The entry command places the generator in swept-frequency mode.

Need checking here

C.4.6 WAIT — Check Instrument Readiness

Syntax:

```
WAIT instrument ready
```

Example:

```
WAIT SPECTRUM ready
```

The `WAIT` command checks whether an instrument has completed an operation.

It uses the readiness parameters defined in the instrument profile.

For example:

```
ready_query=*OPC?
ready_response=+1
ready_timeout_ms=30000
```

These specify the readiness query, expected response, and maximum waiting time.

The supported readiness condition is `ready`.

The controller issues the defined query and checks the response. This is not a continuously repeated polling operation. A readiness check is generally preferable to a fixed delay when the instrument provides a suitable operation-complete query.

C.5 Macro Control Commands

Macro control commands determine the sequence of operations or record information obtained during execution.

C.5.1 DELAY — Pause Execution

Syntax:

```
DELAY milliseconds
```

Example:

```
DELAY 2000
```

The `DELAY` command pauses macro execution for the specified number of milliseconds. In this example, the delay is two seconds.

The supported range is 0 through 60,000 milliseconds per instruction.

A delay does not determine whether an instrument has completed its operation. It simply pauses execution for the requested interval.

C.5.2 SWEEP — Begin a Numerical Loop

Syntax:

```
SWEEP variable start stop increment
```

Example:

```
SWEEP Freq_IN 147MHz 149MHz 100kHz
```

The `SWEEP` command begins a numerical loop.

The first argument identifies the loop variable. The next three specify the starting value, stopping value, and increment.

The loop variable is available to instructions inside the loop using braces.

For example:

```
1=SWEEP Freq_IN 147MHz 149MHz 100kHz
2=SET RFGEN frequency {Freq_IN}
3=WAIT RFGEN ready
4=ENDSWEEP
```

This sequence steps the signal generator through a range of frequencies.

The increment may be positive or negative.

For example:

```
SWEEP Freq_IN 149MHz 147MHz -100kHz
```

specifies a decreasing frequency sweep.

The increment must not be zero and must have the appropriate sign for the requested sweep direction.

The current implementation supports one active SWEEP loop at a time. Nested loops are not supported.

C.5.3 ENDSWEEP — End a Numerical Loop

Syntax:

```
ENDSWEEP
```

The `ENDSWEEP` command marks the end of the instructions repeated by a preceding `SWEEP`.

When execution reaches `ENDSWEEP`, the controller advances the loop variable and either repeats the loop body or continues with the next instruction.

Every `SWEEP` must have a corresponding `ENDSWEEP`. An `ENDSWEEP` without an active loop is invalid.

C.5.4 LOG — Record Measurement Results

Syntax:

```
LOG /logs/filename.csv variable1,variable2,...
```

Example:

```
LOG /logs/frequency_sweep.csv Freq_IN,Freq_Measured,Power_Measured
```

The `LOG` command records the values of the specified runtime variables in a CSV file on the microSD card.

The destination must be a `.csv` file in the `/logs` directory.

The controller creates the file if necessary. When writing to an empty file, it also writes a header containing the variable names. Subsequent records are appended to the file.

For FrequencySweep, the resulting CSV file contains the commanded frequency, measured frequency, and measured amplitude for each sweep point.

These files can be downloaded through the File Management screen and opened in spreadsheet or data-analysis software.

C.6 Variables and Substitution

Macros use variables to transfer values between instructions.

Three kinds of values are commonly encountered:

Input parameters are supplied by the operator through the Macro Panel. They are defined in `[parameter:...]` sections.

Loop variables are created by operations such as `SWEEP`. They change as the macro executes.

Measurement variables are created by queries using the `AS` clause.

All three may be referenced using braces, subject to the restrictions described below.

C.6.1 Input Parameter Substitution

For example:

```
SET RFGEN frequency {start_frequency}
```

replaces `{start_frequency}` with the value entered in the corresponding Macro Panel field.

The controller normalizes numerical, boolean, and selection parameters before substitution.

Thus, a numerical input of `147MHz` is substituted as `147000000` when its base unit is Hz.

C.6.2 Runtime Variables

A runtime variable may be created by a sweep or by an instrument query.

For example:

```
QUERY SPECTRUM marker_frequency AS PeakFreq
```

creates or updates the variable `PeakFreq`.

A subsequent instruction may reference it:

```
SET RFGEN frequency {PeakFreq}
```

Runtime variable names must begin with a letter or underscore. Subsequent characters may be letters, digits, or underscores.

The maximum variable-name length is 31 characters. The current macro execution implementation supports a maximum of 16 runtime variables.

C.6.3 Runtime Arithmetic

The macro language supports simple arithmetic involving runtime variables.

The supported forms are:

```
{variable + constant}

{variable - constant}
```

For example:

```
{Freq_IN + 100kHz}
{Freq_IN - 50kHz}
```

If `Freq_IN` contains 146000000, the first expression produces 146100000, while the second produces 145950000.

The arithmetic is performed during substitution, before the resulting command is executed.

Only one addition or subtraction operation is permitted within an expression.

The left operand must be an existing runtime variable. The right operand must be a numerical constant.

The constant may include one of the supported engineering prefixes: `m`, `k`, `K`, `M`, or `G`.

An alphabetic unit suffix may also be supplied, although the arithmetic evaluator does not perform dimensional checking.

For example:

```
{Freq_IN + 100kHz}
```

is accepted, but the controller does not independently verify that `Freq_IN` represents a frequency.

Multiplication, division, parentheses, and compound arithmetic expressions are not supported.

An important distinction is that arithmetic expressions operate on **runtime variables**, not directly on declared input parameters.

For example, `{Freq_IN + 100kHz}` is supported when `Freq_IN` is a runtime variable. However, an expression such as `{start_frequency + 100kHz}` is not supported merely because `start_frequency` was declared as an input parameter.

C.6.4 Substitution Errors

A macro instruction may fail to expand if it references an undefined parameter or variable.

For example:

```
SET RFGEN frequency {UnknownFrequency}
```

cannot be expanded unless `UnknownFrequency` has been defined and assigned a value.

Similarly, a malformed expression or an expression that produces a value too long for the available buffer causes an error.

Variable and parameter names should therefore be chosen consistently, and the same spelling and capitalization should be used throughout the macro.

C.7 Macro Command Summary

The following table summarizes the supported macro commands.

Command	Syntax	Purpose
`SET`	`SET instrument capability value`	Set an instrument parameter
`QUERY`	`QUERY instrument capability [AS variable]`	Read an instrument parameter

`ACTION`	`ACTION instrument capability`	Execute an instrument action
`ACTIONON`	`ACTIONON instrument capability`	Execute an alternate action
`ENTER`	`ENTER instrument function`	Execute function-entry behavior
`WAIT`	`WAIT instrument ready`	Check instrument readiness
`DELAY`	`DELAY milliseconds`	Pause execution
`SWEEP`	`SWEEP variable start stop increment`	Begin a numerical loop
`ENDSWEEP`	`ENDSWEEP`	End a numerical loop
`LOG`	`LOG /logs/file.csv variables`	Record variables in a CSV file

Square brackets in the QUERY syntax indicate an optional clause. They are not entered literally.

C.8 Macro File Parameters

The following table summarizes the recognized configuration parameters.

Section	Parameter	Purpose
`[macro]`	`description`	Describes the macro
`[macro]`	`name`	Legacy parameter; accepted but ignored
`[parameter:...]`	`type`	`number`, `boolean`, or `select`
`[parameter:...]`	`label`	Displayed parameter label
`[parameter:...]`	`quantity`	Numerical quantity
`[parameter:...]`	`base_unit`	Numerical base unit
`[parameter:...]`	`default`	Default input value
`[parameter:...]`	`options`	Selection choices
`[result:...]`	`label`	Displayed result label
`[result:...]`	`source`	Runtime variable containing result
`[result:...]`	`unit`	Displayed unit; no conversion
`[steps]`	`number=instruction`	Numbered macro instruction

The parser rejects unrecognized parameters within these sections.

C.9 Programming Limits and File Conventions

The current firmware establishes the following limits.

Item	Limit or requirement
Macro filename	Must end in `.cfg`
Macro name from filename	Maximum 79 characters
Macro description	Maximum 159 characters
Input parameters	Maximum 8
Displayed results	Maximum 8

Executable steps	Maximum 32
Step numbering	Consecutive, beginning with 1
Instruction length	Maximum 223 characters
Parameter identifier	Maximum 31 characters
Result identifier	Maximum 31 characters
Runtime variable identifier	Maximum 31 characters
Runtime variables	Maximum 16
`DELAY`	0–60,000 milliseconds
`SWEEP` increment	Nonzero, with correct direction
Nested `SWEEP` loops	Not supported
`LOG` destination	CSV file in `/logs`

C.9.1 Comments and Blank Lines

Blank lines are permitted and may be used to improve readability.

Whole-line comments may begin with either `#` or `;`.

For example:

```
# Configure the signal generator
1=SET RFGEN frequency 146MHz

# Wait for completion
2=WAIT RFGEN ready
```

Comments should be placed on separate lines rather than appended to instructions.

C.9.2 Capitalization and Spacing

Section names, parameter names, command keywords, and identifiers should be entered using the capitalization shown in the examples. In particular, engineering prefixes may be case-sensitive.

The parser removes leading and trailing whitespace from configuration lines and parameter values. Spaces within command instructions are retained.

C.9.3 File Length and Formatting

The parser reads macro files one line at a time using a 256-character input buffer.

Long instructions should therefore be kept within the documented 223-character instruction limit, and long configuration lines should be avoided.

Each numbered instruction must occupy one physical line in the file.

A text editor may visually wrap a line, but it should not insert an actual newline within an instruction.

C.10 Developing and Testing Macros

When developing a new macro, begin with a small number of instructions and verify that they operate correctly before adding loops or more complex measurement procedures.

First, confirm that the instrument identifiers in the macro correspond to entries in `/config/instruments.cfg`.

Next, verify that each referenced capability exists in the corresponding instrument profile.

For example, the instruction:

```
SET RFGEN frequency 146MHz
```

requires an instrument identified as `RFGEN` and a capability named `frequency` containing a valid `set=` definition.

Similarly:

```
ACTION SPECTRUM peak_search
```

requires the spectrum analyzer's profile to contain a capability named `peak_search` with an appropriate action definition.

Once individual operations have been tested, they may be combined into longer sequences.

When an operation requires time to complete, use `WAIT instrument ready` if the instrument supports a suitable readiness query. Otherwise, introduce an appropriate delay.

When using `SWEEP`, begin with a small number of iterations. Confirm that the loop variable changes as expected and that the resulting instrument settings remain within permissible limits.

Finally, verify that measurement variables contain the expected values before using them in subsequent calculations or writing them to CSV files.

The controller performs configuration parsing and preliminary validation before executing a macro. This can identify errors such as invalid section definitions, missing required parameters, excessive numbers of steps, and incorrect step numbering.

Other errors, such as references to unavailable instruments, unsupported capabilities, or invalid instrument responses, may be detected only during command validation or execution.

C.11 Summary

The AK3Y Instrument Controller macro language provides a straightforward method of automating measurements involving GPIB, RS-232, and TCP instruments.

Macros can set instrument parameters, execute actions, retrieve measurements, check readiness, introduce delays, repeat sequences, and record results.

Input parameters allow the operator to modify measurement conditions without editing the macro file, while displayed results provide a convenient way to present measurements in the browser.

Runtime variables and simple arithmetic permit values obtained during execution to be used in subsequent operations.

The two examples in Chapter 5 illustrate these capabilities. FrequencySweep demonstrates repeated measurements involving a signal generator and spectrum analyzer, while FindSignal demonstrates a sequence of analyzer operations that locates and measures the strongest signal within a selected frequency range.

Together, these facilities provide a flexible means of constructing automated measurements without modifying the controller firmware.

Appendix D

Advanced Command Console Reference

D.1 Introduction

The Advanced Command Console provides a means of communicating with instruments directly through the AK3Y Instrument Controller's browser interface, without using the graphical controls displayed in an instrument Control Panel.

The console is located near the bottom of the Instruments screen, below the Workspace Configuration section, as illustrated in Figure 2-1.

It is particularly useful for testing instrument communications, verifying instrument profile definitions, and issuing commands that may not be conveniently accessible through the graphical controls.

The console uses the same instrument definitions and profiles as the Control Panels and macros. Commands are expressed in terms of an instrument identifier and a capability defined in that instrument's profile. The controller translates these instructions into the actual commands required by the instrument and transmits them through the configured GPIB, RS-232, or TCP interface.

Unlike the macro facility described in Chapter 5 and Appendix C, the Advanced Command Console executes one command at a time. It does not provide loops, variables, or automatic recording of measurement data.

D.2 Command Format

All console commands follow a common format:

```
COMMAND instrument capability [value]
```

Here, `COMMAND` identifies the operation to be performed, instrument is the identifier assigned in `/config/instruments.cfg`, and capability identifies a section in the corresponding instrument profile. The optional value is used only with commands that require an argument.

For example:

```
SET RFGEN frequency 146000000
```

This instruction tells the controller to set the frequency capability of the instrument identified as RFGEN to 146,000,000 Hz.

The instrument identifier is the section name in the instrument inventory, not necessarily the descriptive name displayed in the Control Panel. Likewise, the capability is the profile section identifier rather than its displayed label.

The console command names are uppercase and case-sensitive. Instrument and capability identifiers must match their configuration definitions.

Commands are entered in the Command field and executed by selecting **Execute**. The controller displays the execution result and, for queries, the returned response.

D.2.1 Supported Commands

Table D-1. Advanced Command Console Commands

Command	Syntax	Purpose
SET	`SET instrument capability value`	Sets an instrument parameter
QUERY	`QUERY instrument capability`	Queries an instrument parameter
ACTION	`ACTION instrument capability`	Executes a defined action
ACTIONON	`ACTIONON instrument capability`	Executes the alternate action of a toggle control
ENTER	ENTER instrument function	Executes the entry command associated with an instrument function
WAIT	`WAIT instrument ready`	Performs the instrument's readiness check

These six commands constitute the command set implemented by the controller's command dispatcher.

The console does not directly accept arbitrary instrument command strings such as `*IDN?` or `CFRQ?`. Such commands must be associated with capabilities defined in an instrument profile.

D.3 Setting Instrument Parameters — `SET`

The `SET` command transmits a value to an instrument using the command template defined in its instrument profile. The syntax is:

```
SET instrument capability value
```

For example, the Marconi 2024 profile contains:

```
[frequency]
type=number
quantity=frequency
```

```
label=Frequency
base_unit=Hz
set=CFRQ:VALUE {value}
```

Assuming that the Marconi is identified as `RFGEN` in the instrument inventory, entering:

```
SET RFGEN frequency 146000000
```

causes the controller to transmit:

```
CFRQ:VALUE 146000000
```

The controller substitutes the supplied value for the `{value}` placeholder in the profile's `set=` definition.

As another example, entering:

```
SET RFGEN level -20
```

uses the Marconi profile definition:

```
set=RFLV:VALUE {value}DBM
```

to transmit:

```
RFLV:VALUE -20DBM
```

The `value` argument must be entered as a single token without embedded spaces.

Numerical values and engineering units: The Advanced Command Console passes the supplied value directly to the instrument command template. It does not automatically convert engineering prefixes into base units or enforce the `minimum` and `maximum` limits specified in the instrument profile.

Consequently, when setting the Marconi frequency, the operator should enter the frequency in Hz. For example, 146 MHz is entered as `146000000`.

This differs from numerical input fields in the graphical Control Panel, which support engineering-unit entry and conversion. It also differs from macro input parameters, which may be normalized before substitution into a `SET` instruction.

The specified capability must contain a valid `set=` definition with a `{value}` placeholder. If either is missing, the controller reports an error.

D.4 Reading Instrument Parameters — `QUERY`

The `QUERY` command retrieves a value from an instrument using the query command defined in its profile.

The syntax is:

```
QUERY instrument capability
```

For example:

```
QUERY RFGEN frequency
```

The Marconi profile contains:

```
[frequency]
query=CFRQ?
response_prefix=:CFRQ:VALUE
response_terminator=;
```

The controller transmits:

```
CFRQ?
```

The instrument returns its frequency information. The controller then processes the response according to the optional response-prefix and response-terminator definitions in the profile.

For example, suppose the instrument returns:

```
:CFRQ:VALUE 146000000;MODE FIXED
```

The controller locates the specified prefix, extracts the text preceding the semicolon, removes surrounding whitespace, and displays:

```
146000000
```

If the profile contains neither `response_prefix` nor `response_terminator`, the controller returns the instrument response without this additional extraction.

The `QUERY` command therefore provides a convenient way to test both instrument communications and the response-processing definitions in a profile.

Another useful example is:

```
QUERY SPECTRUM identity
```

If the E4407B profile defines:

```
[identity]
type=text
label=Identification
query=*IDN?
```

the controller sends `*IDN?` and displays the instrument's identification response.

`QUERY` requires a valid `query` definition for the specified capability.

D.5 Executing Instrument Actions — `ACTION`

Some instrument operations do not require a numerical value. Examples include initiating a sweep, resetting an operating function, or searching for a signal peak.

These operations are defined in an instrument profile using the `action` parameter.

The console syntax is:

```
ACTION instrument capability
```

For example, the Marconi profile contains:

```
[sweep_reset_action]
type=action
label=Reset Sweep
action=SWEEP:RESET
```

Entering:

```
ACTION RFGEN sweep_reset_action
```

causes the controller to transmit:

```
SWEEP:RESET
```

The `ACTION` command can also execute the primary operation of a `toggle_action` capability.

For example:

```
[sweep_start_action]
type=toggle_action
label=Start Sweep
label_on=Stop Sweep
action=SWEEP:GO
action_on=SWEEP:HALT
```

Entering:

```
ACTION RFGEN sweep_start_action
```

transmits:

```
SWEEP:GO
```

Unlike the graphical toggle button, the console does not automatically alternate between the two commands. The operator explicitly selects the desired operation.

D.6 Executing Alternate Toggle Actions — ACTIONON

The ACTIONON command executes the command specified by the action_on parameter in an instrument profile.

The syntax is:

```
ACTIONON instrument capability
```

Using the Marconi sweep example:

```
ACTIONON RFGEN sweep_start_action
```

causes the controller to transmit:

```
SWEEP:HALT
```

This is the alternate operation associated with the Start Sweep/Stop Sweep control.

ACTION and ACTIONON are independent commands. They do not require the console to maintain the state of the graphical toggle button.

The ACTIONON command requires an action_on definition in the specified capability.

D.7 Entering Instrument Functions — ENTER

The ENTER command executes the instrument command associated with selecting a particular operating function. The syntax is:

```
ENTER instrument function
```

As explained in Appendix B, an instrument profile can define several operating functions, each representing a different collection of controls. For example, the Marconi 2024 profile provides two functions: **Basic RF Generator**, which controls the generator's frequency and RF output level, and **Carrier Frequency Sweep**, which provides controls for swept-frequency operation.

Each function may have an associated entry command that places the instrument in the appropriate operating mode.

D.7.1 Entering the Basic Function

The Basic function is defined implicitly by the [instrument] section of the profile, rather than by a separate type=group section.

For the Marconi 2024, the profile contains:

```
[instrument]
manufacturer=Marconi
```

```
model=2024
description=RF Signal Generator
default_function=Basic RF Generator
default_enter=CFRQ:MODE FIXED
```

The `default_function` parameter specifies the name displayed in the Control Panel's Function selection menu. The `default_enter` parameter specifies the instrument command to be executed when entering this function.

Because the Basic function does not have its own profile section identifier, the controller uses the reserved internal identifier `__basic__` (two underscores before and after `basic`) to refer to it.

Consequently, entering the following command in the Advanced Command Console:

```
ENTER RFGEN __basic__
```

causes the controller to transmit:

```
CFRQ:MODE FIXED
```

This places the Marconi generator in fixed-frequency mode.

The identifier `__basic__` is built into the controller firmware. It is not a parameter that must be added to the instrument profile, and it is not normally visible when using the graphical Control Panel.

D.7.2 Entering Other Instrument Functions

Additional instrument functions are defined explicitly by sections having `type=group`.

For example, the Marconi profile contains:

```
[sweep]
type=group
label=Carrier Frequency Sweep
enter=CFRQ:MODE SWEPT
enter_before_action=true
members=sweep_start,sweep_stop,sweep_step,
sweep_log_step,sweep_time,sweep_mode,sweep_type,
sweep_trigger,sweep_start_action,sweep_reset_action
```

The `[sweep]` section defines the Carrier Frequency Sweep function. Its `label` parameter specifies the name displayed in the Function selection menu, while its `enter` parameter specifies the command used to place the instrument in swept-frequency mode.

To execute this entry command from the Advanced Command Console, enter:

```
ENTER RFGEN sweep
```

The controller transmits:

```
CFRQ:MODE SWEPT
```

Notice that the console uses the profile section identifier `sweep`, rather than the displayed function name Carrier Frequency Sweep.

This convention applies to all functions defined by `type=group` sections.

The `members` parameter identifies the capabilities displayed when the function is selected in the graphical Control Panel. It is not used by the console's `ENTER` command.

Similarly, `enter_before_action=true` controls behavior associated with graphical Control Panel actions; it is not required when the operator explicitly executes an `ENTER` command.

D.7.3 Additional Considerations

The `ENTER` command executes only the entry command associated with the specified function. It does not automatically execute the commands associated with the individual controls belonging to that function.

Furthermore, executing `ENTER` from the Advanced Command Console does not change the function currently selected in an existing graphical Control Panel. It changes the instrument's operating mode by transmitting the specified entry command, but the browser's Function selection remains unchanged.

An entry command is optional. If the specified function does not contain an `enter` parameter, or if the Basic function does not contain `default_enter`, the controller completes the operation without transmitting an instrument command.

In summary, the two forms of the `ENTER` command are:

Function	Console command	Profile parameter used
Basic RF Generator	`ENTER RFGEN __basic__`	`[instrument]` → `default_enter`
Carrier Frequency Sweep	`ENTER RFGEN sweep`	`[sweep]` → `enter`

The distinction is straightforward: the Basic function uses the reserved identifier `__basic__`, while all other functions use their actual profile section identifiers.

D.8 Waiting for Instrument Readiness — `WAIT`

The `WAIT` command performs a readiness check using the synchronization definitions in the instrument profile.

The syntax is:

```
WAIT instrument ready
```

For example:

```
WAIT RFGEN ready
```

The Marconi profile contains:

```
[instrument]
ready_query=*OPC?
ready_response=1
ready_timeout_ms=30000
```

The controller transmits the specified readiness query, `*OPC?`, and waits for a response.

The returned response is compared with the expected text specified by `ready_response`. Leading and trailing whitespace is ignored.

If the response matches, the command completes successfully. Otherwise, the controller reports a synchronization error.

The `ready_timeout_ms` parameter specifies the timeout in milliseconds. The supported range is 1 through 120,000 milliseconds.

The only supported condition for `WAIT` is the literal word `ready`.

For example, `WAIT RFGEN ready` is valid, while `WAIT RFGEN complete` is not.

`WAIT` performs a single readiness query using the configured timeout. It does not repeatedly poll the instrument until the expected response is received.

This distinction is important when constructing automated measurement procedures. The instrument's readiness query should be one that waits for the requested operation to complete or otherwise provides a reliable indication of completion.

D.9 Practical Console Examples

The following examples illustrate how the console can be used to test an instrument profile and verify communication with an instrument.

D.9.1 Testing the Marconi 2024

Assume that the Marconi 2024 is identified as RFGEN in the instrument inventory.

First, enter:

```
ENTER RFGEN __basic__
```

This places the generator in its fixed-frequency operating mode.

Next, enter:

```
SET RFGEN frequency 146000000
```

This sets the output frequency to 146 MHz.

To verify the setting, enter:

```
QUERY RFGEN frequency
```

The controller should display the frequency reported by the instrument.

Next, enter:

```
SET RFGEN level -20
```

This sets the RF output level to −20 dBm.

Finally, enter:

```
QUERY RFGEN level
```

The controller should display the output level reported by the generator.

These operations provide a simple test of the instrument's communications interface and its frequency and RF-level profile definitions.

D.9.2 Testing the E4407B Spectrum Analyzer

Assume that the E4407B is identified as SPECTRUM.

Enter:

```
QUERY SPECTRUM identity
```

The controller transmits `*IDN?` and displays the instrument's identification response.

A numerical capability can be tested in a similar manner.

For example:

```
QUERY SPECTRUM start_frequency
```

retrieves the analyzer's start frequency, provided that the corresponding profile contains a valid query definition.

A setting operation can then be tested using:

```
SET SPECTRUM start_frequency 140000000
```

The profile determines the actual instrument command, including any required units.

The operator should confirm that the requested value is within the instrument's permissible operating range.

D.9.3 Testing a Sweep Action

To test the Marconi's sweep controls, first enter:

```
ENTER RFGEN sweep
```

This selects swept-frequency operation at the instrument.

Next, enter:

```
ACTION RFGEN sweep_start_action
```

The controller transmits the sweep-start command.

To stop the sweep, enter:

```
ACTIONON RFGEN sweep_start_action
```

Finally, enter:

```
ACTION RFGEN sweep_reset_action
```

This transmits the sweep-reset command.

These examples demonstrate how the console can invoke actions that would otherwise be accessed through graphical buttons.

D.10 Console Command Syntax Summary

Table D-2. Command Arguments and Requirements

Command	Required arguments	Profile requirement
`SET`	Instrument, capability, value	Capability must contain `set` with `{value}`
`QUERY`	Instrument, capability	Capability must contain `query`
`ACTION`	Instrument, capability	Capability must contain `action`
`ACTIONON`	Instrument, capability	Capability must contain `action_on`
`ENTER`	Instrument, function	Uses `default_enter` for `__basic__`, or `enter` in the named function section; entry command may be absent

`WAIT`	Instrument, `ready`	`[instrument]` must contain `ready_query`, `ready_response`, and `ready_timeout_ms`

Commands are separated into arguments by spaces or tabs. Extra arguments are not permitted.

The command dispatcher accepts command strings shorter than 160 characters, including the separating spaces. A command that is too long is rejected.

Because the console uses the instrument inventory and profile files, a valid command also requires that the instrument identifier be defined and its associated profile be accessible.

D.11 Error Messages and Troubleshooting

The controller reports the outcome of each console operation.

A successful command normally produces a Success indication. QUERY also displays the response returned by the instrument. When an operation fails, the reported error can help identify the cause.

Table D-3. Common Console Error Messages

Error message	Possible cause
Invalid syntax	Incorrect command name, missing argument, or extra argument
Invalid argument	Required command information is missing or invalid
Configuration error	Instrument inventory or required configuration information is missing
Unsupported interface	Instrument interface is not supported
Invalid address	Invalid GPIB address
Profile error	Instrument profile path or definition cannot be used
Missing command template	Requested capability does not define the required command
Missing `{value}` token	SET template does not contain the required substitution placeholder
Command too long	Command exceeds the supported buffer length
Response buffer too small	Instrument response exceeds the available buffer
Instrument transfer error	Communication with the instrument failed

Instrument synchronization response mismatch	WAIT received a response different from the expected value
Invalid synchronization timeout	WAIT timeout is missing, zero, or outside the supported range
Instrument response parse error	QUERY could not extract the expected response using the profile definitions

For GPIB instruments, communication failures may result from an incorrect address, disconnected cable, or an instrument configured for a different remote-control interface.

For RS-232 instruments, verify the baud rate, data bits, parity, stop bits, and cable wiring. As explained in Section 8.7.2, a null-modem cable or adapter may be required.

For TCP instruments, verify the host address, port number, network connection, and command and response terminators.

If `QUERY` fails while `SET` succeeds, examine the profile's query definition and response-processing parameters.

If `ACTION` or `ACTIONON` fails, confirm that the requested capability contains the corresponding command definition.

When diagnosing a new instrument profile, begin with a simple identification query or another known command before testing more complex operations.

D.12 Relationship to Macros

Several Advanced Command Console commands are also used by the macro language described in Appendix C.

For example, a macro may contain:

```
SET RFGEN frequency {Freq_IN}
```

The macro executor substitutes the current value of the variable `Freq_IN` before the command is executed.

The Advanced Command Console does not provide this macro-variable substitution. The operator must enter an explicit value, such as:

```
SET RFGEN frequency 146000000
```

Similarly, macro commands such as SWEEP, ENDSWEEP, LOG, and variable assignments are handled by the macro executor rather than the console command dispatcher.

The Advanced Command Console is therefore most useful for individual operations and troubleshooting, while macros are intended for automated sequences involving multiple commands, variables, loops, and measurement recording.

D.13 Summary

The Advanced Command Console provides direct access to the controller's profile-driven instrument command system.

Its six supported commands — SET, QUERY, ACTION, ACTIONON, ENTER, and WAIT — allow the operator to set parameters, retrieve values, execute instrument actions, select operating modes, and verify instrument readiness.

All operations use the instrument inventory and profile definitions, so the same command structure can be applied to instruments connected through GPIB, RS-232, or TCP.

The console is particularly valuable when developing or testing instrument profiles, verifying communication with newly connected equipment, and investigating commands that do not behave as expected through the graphical Control Panels.

For automated procedures involving loops, variables, or measurement logging, use the macro facility described in Chapter 5 and Appendix C.